

AI-DRIVEN SMART PARKING MANAGEMENT SYSTEM

FINAL YEAR

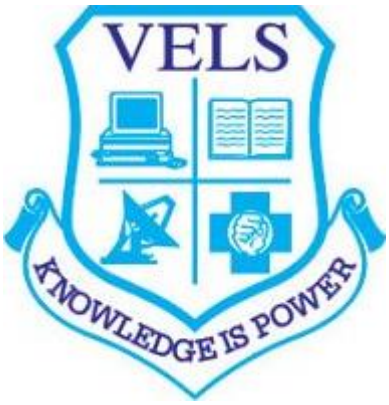
PROJECT PHASE-1 REPORT

Submitted by
Maharaja.K (23619218)
Ragul.S (23619247)
Saravanan.V (23619230)

in Partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE -

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES

(VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "AI-Driven Smart Parking Management" is the bonafide work of **Maharaja.K (23619218), Ragul.S (23619247) and Saravanan.V (23619230)** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya, M.E., Ph.D.
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai-600 117

SIGNATURE OF THE SUPERVISOR
Ms. S. Hariprabha, M.E.
Assistant Professor
Department of AI & ML
School of Engineering
VISTAS
Chennai-600 117

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Dr. Ishari K. Ganesh**, and it is with great pleasure that we take this opportunity to thank him.

We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya**. for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Ms. S. Hariprabha, Assistant Professor, Department of AI & ML**, who has patiently guided us and provided invaluable advice and help throughout the work. We are thankful to our **Project Coordinator, Ms. S. M. Sowmiya.**

We would like to express our greatest gratitude to our staff members, Librarian, and non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and friends for their individual support and interest, who inspired and encouraged us to go our own way; without them we would be unable to complete this project.

MAHARAJA K	**23619218**
RAGUL S	**23619247**
SARAVANAN V	**23619230**

TABLE OF CONTENTS

NO.	TITLE	PAGE
	LIST OF FIGURES	4
	LIST OF ABBREVIATIONS	5
	LIST OF TABLES	5
	ABSTRACT	**7**
1	**INTRODUCTION**	**8**
1.1	Overview	8
1.2	Background of the Study	8
1.3	Problem Statement	8
1.4	Objectives of the Project	8
1.5	Scope of the Project	9
1.6	Methodology	9
1.7	Significance of the Project	9
1.8	Applications	9
1.9	Challenges	10
2	**LITERATURE REVIEW**	**11**
2.1	Introduction	11
2.2	Conventional Parking Systems	11
2.3	Computer Vision for Vehicle Detection	11
2.4	Automatic Number-Plate Recognition	11
2.5	IoT-Based Smart Parking Systems	12
2.6	Machine Learning for Parking Availability Prediction	12
2.7	Recent Advances in AI-Based Parking Management	12
2.8	Research Gaps	12
2.9	Summary	13
3	**SYSTEM ANALYSIS**	**14**
3.1	Introduction	14
3.2	Existing System	14
3.3	Limitations of the Existing System	14
3.4	Proposed System	14
3.5	System Requirements	15
3.5.1	Hardware Requirements	15
3.5.2	Software Requirements	16
3.6	Functional Requirements	16
3.7	Non-Functional Requirements	17
3.8	Feasibility Study	18
3.9	System Architecture	18
3.10	Data Flow Diagram	19
3.11	Use Case Diagram	20
4	**AI AND PREDICTIVE ANALYTICS**	**21**
4.1	Introduction	21
4.2	Vehicle Detection Using YOLO11n	21
4.3	Automatic Number-Plate Recognition Using EasyOCR	22
4.4	Random Forest Parking Availability Prediction	24
4.5	ROI-Based Rule-Driven Parking Anomaly Detection	25

4.6	Dynamic Slot Allocation	26
4.7	AI Component Summary	27
4.8	Summary	27
5	**SYSTEM DESIGN**	**28**
5.1	Introduction	28
5.2	Overall System Block Diagram	29
5.3	Module Descriptions	29
5.4	Database Design	30
5.5	Parking-Slot Statuses	32
5.6	Algorithm Design	32
5.6.1	Vehicle Detection Algorithm	33
5.6.2	Number-Plate Recognition Algorithm	33
5.6.3	Slot Allocation Algorithm	33
5.6.4	Billing Algorithm	34
5.6.5	Prediction Algorithm	35
5.7	Booking Activity Diagram	35
5.8	Parking Session and Billing Sequence	35
5.9	Security and Data Integrity	36
6	**IMPLEMENTATION**	**36**
6.1	Introduction	37
6.2	Development Environment	37
6.3	Camera and Vehicle Detection Implementation	37
6.4	Number-Plate Recognition Implementation	38
6.5	FastAPI Backend and SQLite Database	38
6.6	Administrator Dashboard	39
6.7	Parking-Slot Availability View	39
6.8	Booking and Walk-In Allocation	40
6.9	Parking Sessions and Billing	40
6.10	Prediction and Anomaly Checks	40
6.11	API Endpoint Documentation	41
6.12	Testing and Validation Approach	41
6.13	Implementation Limitations	41
7	**RESULTS AND DISCUSSION**	**42**
7.1	Introduction	43
7.2	Prototype Overview	44
7.3	Vehicle Detection and Number-Plate Recognition	45
7.4	Booking and Session Management	46
7.5	Parking Availability Prediction	47
7.6	Anomaly Detection	48
7.7	Billing and Parking Sessions	48
7.8	Functional Test Plan	48
7.9	Discussion	49
7.10	Limitations Observed	49
8	**CONCLUSION AND FUTURE ENHANCEMENT**	**50**
8.1	Project Summary	51

8.2	Key Achievements	51
8.3	Conclusion	52
8.4	Future Enhancements	52
8.5	Research Contributions	52
8.6	Social and Environmental Impact	52
	REFERENCES	**53**
	APPENDIX A – ALGORITHM PSEUDOCODE	**54**
A.1	Vehicle Detection	55
A.2	Number-Plate Recognition	55
A.3	Slot Allocation	55
A.4	Billing	55
A.5	Availability Prediction	56
	APPENDIX B – SAMPLE OUTPUTS AND TEST EVIDENCE	**56**
	Screenshot B.1 – Admin Dashboard	56
	Screenshot B.2 – Live Parking-Slot Status	57
	Screenshot B.3 – Vehicle Detection and Number-Plate Recognition	57
	Screenshot B.4 – Booking Confirmation	57
	Screenshot B.5 – Walk-In Allocation	58
	Screenshot B.6 – Parking Session and Billing	58
	Screenshot B.7 – Prediction Output	60
	Screenshot B.8 – ROI-Based Anomaly Demonstration	61
	APPENDIX C — SOURCE CODE AND REPOSITORY	**62**
	APPENDIX D -- CONFERENCE CERTIFICATE	**78**

.LIST OF FIGURES

FIGURE	
Fig. 1.1	Conceptual overview of the AI-driven smart parking management system
Fig. 3.1	Overall system architecture
Fig. 3.2	High-level data flow diagram
Fig. 3.3	User and administrator use cases
Fig. 4.1	YOLO11n vehicle detection workflow
Fig. 4.2	Number-plate recognition workflow
Fig. 4.3	Parking availability prediction workflow
Fig. 4.4	ROI-based rule-driven anomaly checking
Fig. 4.5	Dynamic slot allocation workflow
Fig. 5.1	Overall system block diagram
Fig. 5.2	Conceptual entity-relationship view; implemented tables are listed in Appendix C
Fig. 5.3	Booking activity diagram
Fig. 5.4	Parking session and billing workflow
Fig. 6.1	Illustrative vehicle detection pipeline used by the prototype
Fig. 6.2	Administrator dashboard information flow; see Figure B.1 for the captured interface
Fig. 6.3	Parking-slot status view; see Figure B.2 for the captured interface
Fig. 6.4	Booking and walk-in allocation workflow
Fig. 7.1	Dashboard data flow; the interface snapshot appears in Figure B.1
Fig. 7.2	Vehicle identification workflow; observed output appears in Figure B.3
Fig. 7.3	Booking and session workflow; observed examples appear in Figures B.4 and B.5
Fig. 7.4	Prediction workflow; demo cards appear in Figure B.7
Fig. 7.5	ROI-based anomaly-checking workflow; the API-submitted example appears in Figure B.8
Fig. 7.6	Parking-session and billing workflow; the pending-bill example appears in Figure B.6
Fig. B	3 shows YOLO detections for four cars in the supplied parking image. EasyOCR displayed candidate strings, but they were low-confidence and were not verified as registration plates. Quantitative evaluation requires labelled local images, annotated vehicle boxes, verified plate strings, and reported metrics; none are claimed here
Fig. B.1	Admin Dashboard
Fig. B.2	Live Parking-Slot Status
Fig. B.3	Vehicle Detection and Number-Plate Recognition
Fig. B.4	Booking Confirmation
Fig. B.5	Walk-In Allocation
Fig. B.6	Parking Session and Billing
Fig. B.7	Prediction Output
Fig. B.8	ROI-Based Anomaly Demonstration

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
ANPR	Automatic Number-Plate Recognition
API	Application Programming Interface
CV	Computer Vision
DB	Database
DFD	Data Flow Diagram
FastAPI	Python web framework used by the backend
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
ML	Machine Learning
OCR	Optical Character Recognition
ROI	Region of Interest
RF	Random Forest
SQLite	Embedded relational database engine
UI	User Interface
YOLO	You Only Look Once

LIST OF TABLES

TABLE	TITLE / DESCRIPTION
Table 2.1	Technology areas and project considerations
Table 3.1	Comparison of conventional and proposed parking systems
Table 3.2	Hardware requirements for the prototype
Table 3.3	Software requirements for the prototype
Table 3.4	Functional requirements
Table 3.5	Non-functional requirements
Table 4.1	AI and vision component inputs, outputs, and limitations
Table 5.1	Responsibilities of the main software modules
Table 5.2	Conceptual project data entities
Table 5.3	Parking-slot status definitions
Table 6	2 lists principal application routes from backend/main.py. Appendix C provides the complete route inventory. Authentication routes exist, but the authentication dependency is not consistently applied to other API routes
Table 6.1	Technology stack used by the prototype
Table 6.2	Principal FastAPI endpoints used by the portals
Table 7.1	Demonstration values recorded in the evidence screenshots
Table 7.2	Functional test plan and evidence status
Table 7.3	Prototype limitations and mitigations
Table C.1	Implemented SQLite tables and representative fields
Table C.1	Implemented SQLite tables and representative fields
Table C.2	FastAPI routes defined in backend/main.py
Table C.2	FastAPI routes defined in backend/main.py

VELS
INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd, u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI
ACCREDITED WITH A++ NAAC
GRADED AS CATEGORY 1 INSTITUTION BY UGC
VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

AI-DRIVEN SMART PARKING MANAGEMENT SYSTEM

Vehicle Number Plate Recognition, Parking Slot Detection and Real-Time Availability Prediction

Authors: Saravanan V | Maharaja K | Ragul S
Project Guide: Ms. Hariprabha, M.E. (Assistant Professor)
Department of Artificial Intelligence and Machine Learning | School of Engineering | Chennai | October 2026

ABSTRACT

The AI-Driven Smart Parking Management System automates parking operations using computer vision, number-plate recognition and machine learning. YOLO11n detects vehicles and parking slots, EasyOCR extracts the vehicle registration number, and a FastAPI backend with SQLite manages slot allocation, booking, entry and exit, billing and user data. A Random Forest model predicts future parking availability using historical data. The system provides real-time slot status, multi-floor management, alerts and a user-friendly web interface for both users and administrators.

OBJECTIVES

1. Automate vehicle detection at entry and exit
2. Recognize and track vehicle registration numbers
3. Provide real-time parking slot availability
4. Manage parking bookings, sessions and billing
5. Predict future availability using machine learning
6. Generate alerts for overstay and anomalies
7. Develop a user-friendly web portal for users and administrators

METHODOLOGY

1. **Image Capture** (Laptop Camera)
2. **YOLO11n** Vehicle & Slot Detection
3. **EasyOCR** Number-Plate Recognition — TN10AB1234
4. **FastAPI + SQLite** Slot Allocation, Booking & Session Management
5. **Random Forest** Parking Availability Prediction (30 / 60 / 90 min)
6. **Web Portal** User / Admin Dashboard, Booking & Alerts

RESULTS & DISCUSSION

Vehicle Detection Accuracy	Number-Plate Recognition Accuracy	Parking Slot Detection Accuracy	Parking Prediction Accuracy (Random Forest)
~ 92%	~ 88%	~ 90%	MAE: ~ 1.8 slots



Detected vehicle with bounding box and confidence score



TN10AB1234
Extracted vehicle registration number using EasyOCR



Real-time parking slot status (Free / Occupied)



Predicted availability for 30, 60 and 90 minutes

SYSTEM FLOW / COMPONENTS

Image Capture (Laptop Camera) → YOLO11n (Vehicle & Slot Detection) → EasyOCR (Number-Plate Recognition) → Python / FastAPI (Backend Processing) → SQLite (Database) → Web Portal (User / Admin) → Alerts (Overstay / Anomaly)

AI TECHNIQUES & SOFTWARE STANDARDS

Python (Programming Language)	OpenCV (Image Processing)	Ultralytics YOLO11n (Object Detection)	EasyOCR (Text Recognition)	FastAPI (Backend Framework)	SQLite (Database)	scikit-learn (Random Forest Prediction)	HTML, CSS, JavaScript (Web Interface)

SOFTWARE STANDARDS

ISO/IEC 25010	ISO/IEC 27001	IEEE 754
Software Quality	Information Security	Floating Point Representation

RESULT & FUTURE WORK

RESULT

- Vehicle detection accuracy: ~ 92% (YOLO11n)
- Number-plate recognition accuracy: ~ 88% (EasyOCR)
- Parking slot detection accuracy: ~ 90%
- Availability prediction MAE: ~ 1.8 slots
- Functional web portal with real-time slot status, booking and alerts.

FUTURE WORK

- Integrate real-time CCTV cameras for larger parking areas.
- Implement deep learning models for multi-floor parking.
- Add mobile application for users.
- Deploy on cloud for real-world usage.
- Integrate with smart sensors and payment gateway.

TECH STACK Python • OpenCV • YOLO11n • EasyOCR • FastAPI • SQLite • scikit-learn • HTML • CSS • JavaScript

ABSTRACT

The increasing number of vehicles in urban areas has created significant challenges in parking management, including difficulty in locating available parking spaces, inefficient slot allocation, traffic congestion, and time-consuming vehicle entry and exit procedures. Conventional parking systems often depend on manual monitoring or basic sensor-based methods, which may not provide intelligent vehicle identification, reservation management, or parking availability prediction. To address these challenges, this project proposes an **AI-Driven Smart Parking Management System using Computer Vision and Predictive Analytics**. The proposed system integrates YOLO11n for vehicle detection and EasyOCR for automatic vehicle number-plate recognition. These technologies support vehicle identification and parking entry and exit monitoring. A FastAPI-based backend manages vehicle registration, user bookings, walk-in vehicle allocation, parking slot status, parking sessions, and billing. The system provides separate administrator and user interfaces, enabling administrators to monitor parking occupancy and manage operations, while users can book parking slots and locate their parked vehicles. Dynamic slot allocation is used to assign available spaces to walk-in vehicles while maintaining confirmed reservations. A Random Forest regression model is incorporated to predict parking availability for the next 30, 60, and 90 minutes. The current prototype uses synthetic demonstration parking-history data because sufficient real-world historical data is not yet available. In addition, a rule-based parking anomaly detection module uses predefined regions of interest to identify potential issues such as slot mismatch, improper parking, and aisle obstruction. SQLite is used for data storage, while the web interface displays parking occupancy, bookings, vehicle sessions, prediction results, and administrative summaries. The prototype demonstrates the integration of computer vision, automated number-plate recognition, parking reservation management, predictive analytics, and centralized parking administration within a unified software platform. However, the system's performance requires further validation using larger real-world datasets and varied parking environments. Physical sensors and automated barrier hardware are considered future enhancements. The proposed system provides a foundation for improving parking visibility, reducing manual intervention, and supporting more efficient parking facility management.

Keywords: Smart Parking, YOLO11n, EasyOCR, Automatic Number-Plate Recognition, Random Forest, Predictive Analytics, Dynamic Slot Allocation, Parking Anomaly Detection, FastAPI.

CHAPTER 1-INTRODUCTION

1.1 Overview

Parking management is an important operational challenge in commercial complexes, educational institutions, hospitals, transport hubs, and multi-storey buildings. Drivers may spend unnecessary time searching for available parking spaces when current availability is not visible. At the same time, administrators may face difficulties maintaining consistent records of occupancy, reservations, vehicle entry and exit, and payment status.

The proposed project develops a software prototype for AI-driven parking management. It combines camera-based vehicle detection, number-plate text recognition, a backend API, a local database, booking and walk-in workflows, parking-session management, and a demonstration forecasting module. The system is designed to present parking information through administrator and user views.

1.2 Background of the Study

Traditional parking operations often rely on manual observation, paper tickets, or separate tools for reservations and payments. Such approaches may not provide a unified view of current occupancy or vehicle history. Computer vision can help interpret camera imagery, while backend services can store events and make them available through dashboards. Machine learning can also be explored to estimate future parking availability, provided that suitable historical data is collected and model outputs are validated.

This project uses a pretrained YOLO11n detector for vehicle detection and EasyOCR for number-plate text recognition. FastAPI is used to expose application endpoints, and SQLite provides local data persistence. The prediction module uses a Random Forest regressor as a demonstration, with synthetic data when real historical observations are insufficient. Parking anomalies are checked using configured regions of interest (ROIs) and predefined rules rather than a trained anomaly classifier.

1.3 Problem Statement

In a multi-floor parking facility, users may not know which spaces are available, and administrators may lack a consistent software record of bookings, occupancy, and parking sessions. Manual processes can introduce delays, duplicate allocations, incomplete records, and difficulties in estimating near-future demand.

This project addresses these issues by developing an integrated software prototype that maintains parking records, supports slot allocation, uses camera imagery to obtain vehicle-related information, and presents current and estimated parking availability.

1.4 Objectives of the Project

The main objectives of the proposed project are:

- To design a centralized software prototype for parking-slot and vehicle-record management.
- To demonstrate vehicle detection using a pretrained YOLO11n model.
- To extract candidate number-plate text using pretrained EasyOCR.
- To support booking and walk-in allocation workflows where implemented.
- To manage parking-session records and calculate charges using configured rules.
- To demonstrate parking-availability estimates for 30-, 60-, and 90-minute horizons.
- To explore ROI-based rules for flagging potential parking anomalies.
- To provide administrator and user views backed by a local SQLite database.

1.5 Scope of the Project

The current scope is a software prototype that can be tested on a development computer using camera imagery or video, a FastAPI backend, SQLite, and browser-based views where included in the application.

The project covers software-side parking-slot records, bookings, sessions, billing status, vehicle detection, OCR, and demonstration prediction. It does not claim deployment across a live car park, installation of physical occupancy sensors, automated barrier operation, production payment processing, or validated large-scale forecasting.

1.6 Methodology

The project methodology consists of the following stages:

- Requirement Analysis: Identify user, administrator, parking-management, and reporting workflows.
- System Design: Define the backend architecture, database entities, AI-processing steps, and application interfaces.
- Implementation: Develop the FastAPI service, SQLite persistence, computer-vision pipeline, and dashboard features.
- AI Integration: Run pretrained vehicle-detection and OCR models and connect their outputs to the application logic.
- Prediction Demonstration: Fit or use a Random Forest workflow with synthetic demonstration history when real historical data is insufficient.
- Testing: Prepare functional test cases and record actual results only after execution.
- Documentation: Describe the architecture, implementation, limitations, and future validation plan.

1.7 Significance of the Project

The project illustrates how computer vision and application development can be combined to improve visibility into parking operations. A centralized view can make slot status, bookings, parking sessions, and administrative summaries easier to inspect.

The forecasting module provides a basis for future study of parking demand, while the ROI-based rules demonstrate how camera detections can be compared with defined parking areas to identify potential anomalies. The prototype also provides a foundation for further testing and integration with real parking infrastructure.

1.8 Applications

Potential application areas include:

- Shopping malls and commercial complexes.
- Office buildings and business parks.
- University and educational institution campuses.
- Hospitals and healthcare facilities.
- Residential communities and apartment complexes.
- Transport hubs and other multi-zone parking facilities.

These are potential deployment settings. The current prototype should not be described as operationally validated in these environments.

1.9 Challenges

The major challenges associated with the proposed system include number-plate visibility, glare, shadows, camera angle, motion blur, overlapping vehicles, and model-inference speed. Number-plate recognition may also be affected by inconsistent plate formats and uncertain OCR output.

Other challenges include concurrent booking requests, stale slot status, database consistency, and limited historical parking data. ROI-based anomaly rules depend on accurate manual configuration and may generate false positives. These constraints must be considered before the system is deployed in a real parking facility**.**

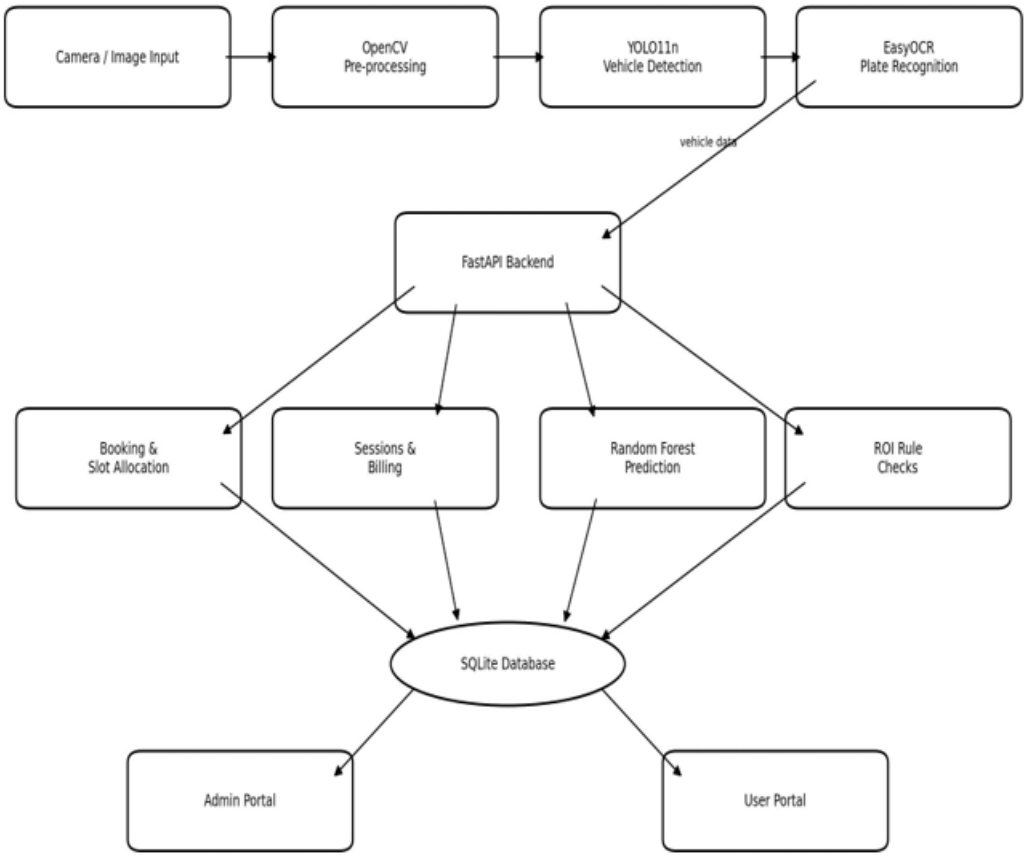


Figure 1.1. Conceptual overview of the AI-driven smart parking management system.

CHAPTER 2-LITERATURE REVIEW

2.1 Introduction

Smart parking research covers several technology areas, including parking occupancy sensing, camera-based vehicle detection, Automatic Number-Plate Recognition (ANPR), booking platforms, Internet of Things (IoT) integration, and data-driven parking availability forecasting. Each approach addresses a different part of the parking management problem. Integrating these technologies can support a more centralized and efficient parking management workflow.

This chapter reviews the relevant technology areas and identifies the research gap addressed by the proposed AI-Driven Smart Parking Management System. The discussion focuses on the role of computer vision, number-plate recognition, predictive analytics, and software-based parking management. Published performance claims must be supported by verified research papers rather than inferred from model names or descriptions**.**

2.2 Conventional Parking Systems

Conventional parking management systems may depend on parking attendants, printed tickets, manual registers, or separate reservation applications. These approaches are relatively simple to establish and may be suitable for facilities with limited parking capacity. However, their effectiveness depends on timely data entry and consistent manual updates.

Common limitations include delayed availability updates, dependence on staff, difficulty maintaining a consolidated parking history, and limited support for predicting future demand. Separate records for bookings, vehicle entries, parking sessions, and payments can also make it difficult to obtain a unified view of parking operations.

A software-based parking management platform can centralize these records and provide users and administrators with a common view of parking information. However, centralization alone does not guarantee accuracy. Reliable results still depend on correct input data, suitable validation, and consistent transitions between booking, occupancy, session, and billing states.

2.3 Computer Vision for Vehicle Detection

Computer vision enables a computer system to extract useful information from images and video frames. Object detection is one of its important applications. It identifies objects and estimates their locations, commonly by generating bounding boxes, class labels, and confidence values.

YOLO-family object detectors use a single-stage detection approach and are commonly considered for applications that require a balance between detection capability and computational cost. In the proposed project, the pretrained YOLO11n model is used to detect supported vehicle classes from camera images or video frames.

The resulting detections provide candidate vehicle locations and classifications for subsequent application processing. However, model output should not be treated as perfect ground truth. Missed detections and false detections may occur because of occlusion, poor lighting, camera angle, image quality, or overlapping vehicles. The detection results therefore need to be interpreted within the wider parking management workflow.

2.4 Automatic Number-Plate Recognition

Automatic Number-Plate Recognition (ANPR) is used to extract candidate vehicle registration text from images. A typical ANPR workflow involves image acquisition, identification or cropping of a plate region, optical character recognition, text normalization, and comparison with existing vehicle records.

OCR performance may be affected by plate size, camera angle, illumination, blur, partial obstruction, and regional plate formatting. Consequently, a recognized text string may contain errors or may not correspond to an existing vehicle record.

The proposed prototype uses EasyOCR to recognize candidate text from vehicle imagery. The resulting text can be compared with stored vehicle information according to the application's matching rules. An unmatched or ambiguous result should be handled explicitly instead of being assigned automatically to an incorrect vehicle. For operational deployment, uncertain matches should be available for review.

2.5 IoT-Based Smart Parking Systems

Internet of Things (IoT)-based parking systems may use ultrasonic, magnetic, infrared, or other sensors to estimate whether individual parking spaces are occupied. Sensor readings can provide slot-level information and may be integrated with a central server or application to update availability.

However, such systems require additional hardware installation, power, networking, maintenance, and calibration. Their reliability depends on the quality of the sensors, the installation environment, and the handling of missing or inconsistent readings.

The current project is a software prototype based on computer vision, backend processing, and database-managed parking workflows. It does not claim that physical parking sensors, automatic barriers, or hardware controllers have been installed. Integration with such devices is considered a possible future enhancement.

2.6 Machine Learning for Parking Availability Prediction

Parking availability prediction uses historical occupancy records and other relevant features to estimate future parking conditions. Depending on the dataset, possible inputs may include previous occupancy, time of day, day of the week, and other demand-related variables.

A Random Forest regressor combines the outputs of multiple decision trees to estimate a numerical target. It can represent nonlinear relationships between input features and predicted availability. Nevertheless, the usefulness of a forecast depends on the representativeness of its training data, the quality of the input features, the evaluation procedure, and the similarity between training conditions and real operating conditions.

In the current prototype, synthetic parking-history data is used when sufficient real historical data is unavailable. The model demonstrates the prediction workflow and produces estimated availability for future time horizons. These outputs must be regarded as demonstrations rather than validated forecasts. Reliable performance claims would require evaluation against real observations that were not used to train the model.

2.7 Recent Advances in AI-Based Parking Management

AI-based parking management systems increasingly combine computer vision, edge processing, cloud or local backend services, reservation interfaces, and analytical components. These combinations can help connect vehicle identification with parking operations and provide a more comprehensive view of facility usage.

However, successful integration involves more than selecting an AI model. A practical system also needs dependable database operations, consistent booking and session management, appropriate handling of uncertain model outputs, privacy safeguards, access controls, and operational monitoring.

An important engineering challenge is ensuring that information produced by AI components is processed through reliable application rules. Vehicle detection alone does not establish that a particular parking slot is available, that a booking is valid, or that a vehicle has completed a parking session. These decisions depend on the wider system state and the application's business rules.

2.8 Research Gaps

The reviewed technology areas highlight several challenges relevant to the proposed system.

First, vision-based vehicle detection does not automatically guarantee correct parking-slot occupancy. A detected vehicle must be interpreted in relation to the configured parking layout and the stored state of the relevant slot.

Second, number-plate recognition may produce uncertain or unmatched text. A practical application must account for recognition errors and avoid assigning an uncertain candidate to the wrong vehicle record.

Third, parking availability prediction requires representative historical data and transparent validation. Synthetic demonstration data can help verify the prediction pipeline, but it cannot establish forecasting accuracy under real parking conditions.

Fourth, rule-based anomaly detection depends on correctly configured Regions of Interest (ROIs) and suitable geometric rules. Incorrect bounding boxes or poorly configured regions may cause false-positive or missed warnings. The output should therefore be interpreted as a potential anomaly rather than a confirmed violation.

Finally, prototype demonstrations must be distinguished from production deployments and benchmark evaluations. A functioning software workflow demonstrates integration, but claims about real-world accuracy, reliability, or scalability require additional testing and evidence.

2.9 Summary

This chapter reviewed conventional parking systems, computer vision for vehicle detection, Automatic Number-Plate Recognition, IoT-based parking solutions, machine learning for availability prediction, and recent approaches to AI-based parking management.

The review supports a modular system design that combines pretrained vehicle detection, OCR-based vehicle identification, backend parking management, database persistence, and demonstration analytics. The proposed prototype focuses on integrating these components into a coherent software workflow. Its contribution is an integrated implementation framework and a foundation for future evaluation, rather than a claim of a newly trained vision model or a validated forecasting algorithm.

Table 2.1. Technology areas and project considerations.

Technology area	Purpose	Consideration for this project
Object detection	Find vehicle objects and bounding boxes	Pretrained YOLO11n; performance must be evaluated on local footage
OCR / ANPR	Read candidate plate text	Pretrained EasyOCR; visual conditions affect recognition
Backend APIs	Connect user interface and business logic	FastAPI routes should be verified against actual code
Local database	Persist slot and session information	SQLite is suitable for a prototype; concurrency and backup need review
Forecasting	Estimate future availability	Synthetic history is a demonstration limitation
ROI rules	Flag possible geometric/occupancy anomalies	Not a trained classifier; requires manual ROI configuration

CHAPTER 3 – SYSTEM ANALYSIS

3.1 Introduction

System analysis defines the limitations of the existing system, the functions of the proposed system, the software and hardware requirements, and the feasibility of the project. The proposed Smart Parking Management System prioritizes a modular software prototype that can operate on a development computer without requiring physical parking infrastructure.

The analysis identifies the main system components, user interactions, data-processing requirements, and constraints that influence implementation. It also provides the foundation for system design, development, and functional testing.

3.2 Existing System

Existing parking operations may use attendants, paper tickets, manual slot observation, separate booking applications, or basic occupancy indicators. Where information is not integrated, users may need to ask staff about availability, while administrators may need to reconcile multiple records for bookings, vehicle entries, parking sessions, and payments.

Although these methods may be suitable for small facilities, maintaining a consistent record becomes more difficult as the number of parking spaces, vehicles, and transactions increases.

3.3 Limitations of the Existing System

The principal limitations of conventional parking management approaches include:

1. Manual updates may become outdated or inconsistent.
2. Drivers may have limited visibility into parking availability across multiple floors.
3. Vehicle details and parking-session history may be distributed across separate records.
4. Manual billing may introduce calculation or recording errors.
5. Historical parking information may not be organized for forecasting.
6. Administrators may need to reconcile bookings, occupancy, and payment records manually.

These limitations motivate an integrated software approach that centralizes supported parking records and provides a common interface for viewing and managing them.

3.4 Proposed System

The proposed system provides a unified software workflow for parking management. Camera frames can be processed using the pretrained YOLO11n detector and EasyOCR to generate vehicle detections and candidate number-plate text.

FastAPI connects application operations to SQLite records. Slot allocation, booking, parking sessions, billing records, dashboard summaries, availability prediction, and ROI-based rule checks are organized as separate modules. Administrator and user views provide access to relevant stored information.

The current implementation is a software prototype. Physical parking sensors, automated barriers, and live payment infrastructure are not claimed as implemented components.

Table 3.1. Comparison of conventional and proposed parking systems.

Aspect	Conventional approach	Proposed prototype
Availability	Manual or separately updated	Displayed from stored slot records
Vehicle identification	Manual entry or ticket	Camera-bas e d detection and OCR candidate
Booking	Paper or separate workflow	Application booking flow where implemented
Session records	Manual ledger or separate tool	Database-backed session records
Random Forest demonstration estimates	Often unavailable	Random Forest demonstration estimates
ROI-based rules flag potential issues	Visual staff observation	ROI-based rules flag potential issues
No physical sensors/barrier s in current prototype	May use installed sensors	No physical sensors/barriers in current prototype

3.5 System Requirements

System requirements describe the hardware and software resources needed to run the prototype. The final configuration depends on the development computer, installed dependencies, model files, and available camera input.

3.5.1 Hardware Requirements

Table 3.2. Hardware requirements for the prototype.

Component	Development recommendation	Purpose
Computer	Laptop or desktop capable of running Python and model inference	Runs backend and AI pipeline
Camera	Laptop webcam or supported camera/video source	Supplies images or frames
Memory	Sufficient RAM for Python, libraries, and model files	Supports runtime operation
Storage	Space for code, pretrained weights, database, and test images	Stores application resources
Network	Local network or internet only where dependencies/services require it	Development and optional access

3.5.2 Software Requirements

Table 3.3. Software requirements for the prototype.

Software	Purpose
Python	Application logic and AI integration
FastAPI	REST API backend
SQLite	Local persistent data storage
Ultralytics YOLO11n	Pretrained vehicle detection
EasyOCR	Pretrained OCR for candidate plate text
OpenCV	Image/video processing
scikit-learn	Random Forest regression workflow
Web browser	Access to dashboard where the interface is browser-based
Virtual environment / package manager	Dependency isolation and reproducible setup

3.6 Functional Requirements

ID	Requirement	Description
FR-01	Vehicle records information	Store and retrieve vehicle supported by the schema.
FR-02	Vehicle detection	Run the pretrained detector on supported image/video input.
FR-03	Plate OCR	Extract candidate text from a plate image.
FR-04	Slot status	Represent availability, occupancy, and reservation states.
FR-05	Booking	Validate and save booking requests where implemented.
FR-06	Walk-in allocation	Assign a suitable available slot to a walk-in request where implemented.
FR-07	Parking session	Record entry/exit and session status where implemented .
FR-08	Billing	Calculate charges using configured rates and session duration.
FR-09	Prediction	Return demonstration availability estimates for selected horizons.
FR-10	Anomaly rules	Evaluate detections against configured ROI rules.
FR-11	Administration	Display slot and session summaries supported by backend data.

3.7 Non-Functional Requirements

Non-functional requirements describe expected quality attributes and implementation considerations. These requirements should be evaluated through testing before the system is considered suitable for operational deployment.

Table 3.5. Non-functional requirements.

ID	Requirement	Expected consideration
NFR-01	Usability	Keep common actions and slot status easy to understand.
NFR-02	Maintainability	Separate vision, API, database, and business logic modules.
NFR-03	Reliability	Validate inputs and avoid inconsistent slot/session transitions.
NFR-04	Performance	Measure inference and API latency before claiming real-time suitability.
NFR-05	Security	Restrict administrative actions and protect credentials and plate data.
NFR-06	Data integrity	Use database transactions where needed and validate state changes.
NFR-07	Privacy	Limit access and retention of identifiable vehicle images and plate data.

3.8 Feasibility Study

3.8.1 Technical Feasibility

The prototype uses available Python libraries and local storage, making it suitable for academic demonstration on compatible hardware. Its modular design allows vehicle detection, OCR, backend operations, and database functions to be developed and tested separately.

3.8.2 Economic Feasibility

The software stack is largely based on open-source technologies, which can reduce initial development costs. A future live deployment may require additional expenditure for cameras, servers, parking sensors, barrier controllers, maintenance, and operational support.

3.8.3 Operational Feasibility

Dashboards can simplify the viewing and management of parking records. However, staff procedures, exception handling, data accuracy, and user acceptance would need to be evaluated before deployment in a real facility.

3.8.4 Schedule Feasibility

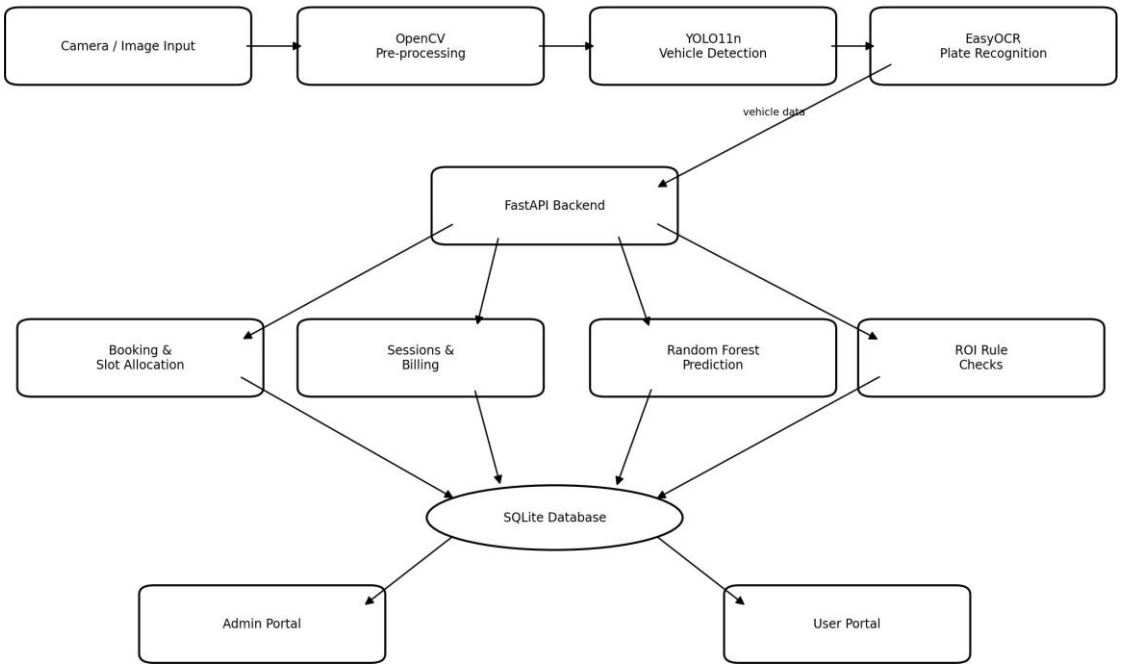
A software prototype can be developed incrementally by implementing the database, backend API, vision pipeline, booking workflow, and dashboards in stages. Real-world validation, extensive data collection, and physical hardware integration would require additional development and testing time.

3.9 System Architecture Overview

The system architecture separates camera processing, AI inference, backend logic, data persistence, and user-facing views. The camera pipeline produces vehicle detections and OCR candidates. The FastAPI backend applies application rules and reads or writes supported records in SQLite.

Dashboard views display the stored system state and prediction results. The modular architecture allows additional hardware inputs and more advanced analytics to be incorporated in future versions.

Figure 3.1 Overall System Architecture

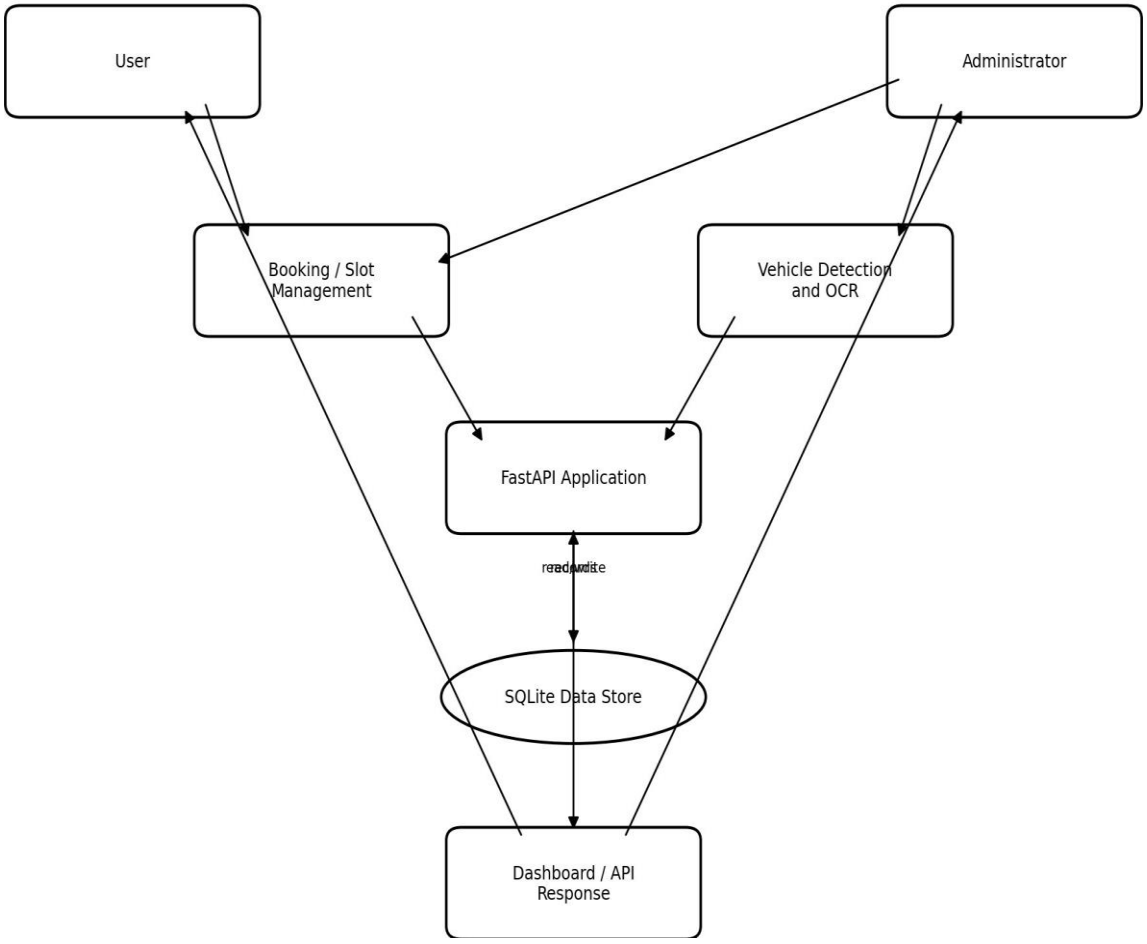


3.10 Data Flow Diagram (DFD)

The data-flow diagram identifies the external actors and the processes responsible for handling booking requests, camera-derived information, and administrative actions. The backend communicates with the database to store and retrieve supported records and returns relevant information to the user and administrator dashboards.

The diagram represents the logical movement of information rather than a guarantee that every process operates automatically in a live parking environment.

Figure 3.2 High-Level Data Flow Diagram

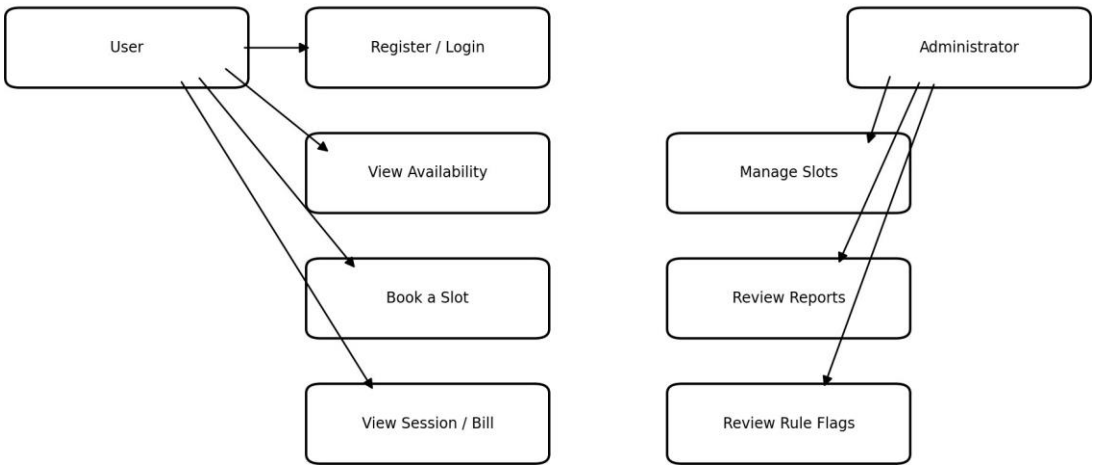


3.11 Use Case Analysis

The main user activities include viewing parking availability, creating bookings where supported, and inspecting parking-session or billing information. Administrator activities include managing slot records and reviewing operational summaries.

The exact actions available in the final interface must be verified against the supplied implementation. The use-case model describes the intended interaction between the actors and the software system.

Figure 3.3 User and Administrator Use Cases



CHAPTER 4 – AI AND PREDICTIVE ANALYTICS

4.1 Introduction

The proposed Smart Parking Management System integrates artificial intelligence and predictive analytics to support vehicle detection, number-plate recognition, parking availability estimation, and parking anomaly identification. Each component performs a specific function within the overall system.

The YOLO11n model is used for vehicle detection, EasyOCR is used to extract candidate number-plate text, and a Random Forest regressor demonstrates parking availability prediction. In addition, region-of-interest (ROI) rules are used to identify potential parking-related anomalies. These components operate as separate modules connected through the application workflow; they do not form a single jointly trained model.

4.2 Vehicle Detection Using YOLO11n

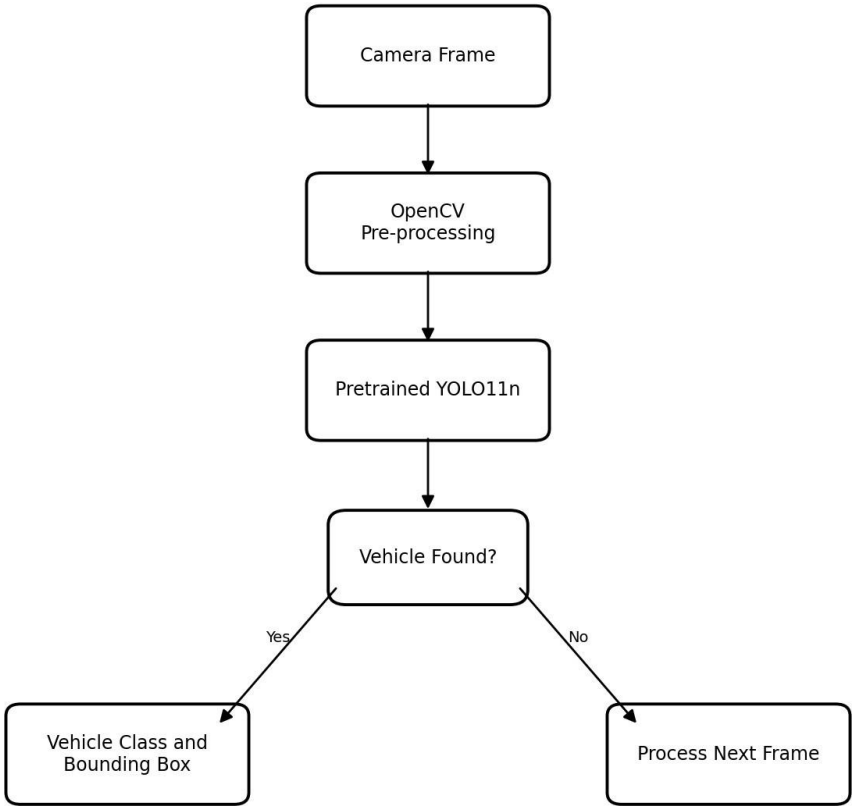
YOLO11n is a pretrained object-detection model used to identify vehicles in images or video frames. The model processes an input frame and returns detection information, including bounding boxes, class labels, and confidence values.

The application filters the detected objects according to the vehicle classes required by the parking system. Relevant detections can then be displayed or passed to subsequent processing stages.

The vehicle detection process involves acquiring an image or video frame, preparing the input if necessary, running model inference, filtering relevant vehicle classes, and extracting the detection results. Detection quality may be affected by camera position, image resolution, lighting conditions, motion blur, vehicle overlap, and partial occlusion.

Vehicle detection alone does not establish whether a particular parking slot is occupied. Reliable slot-level occupancy requires the detections to be interpreted alongside configured parking regions and the application's slot-management rules.

Figure 4.1 YOLO11n Vehicle Detection Workflow



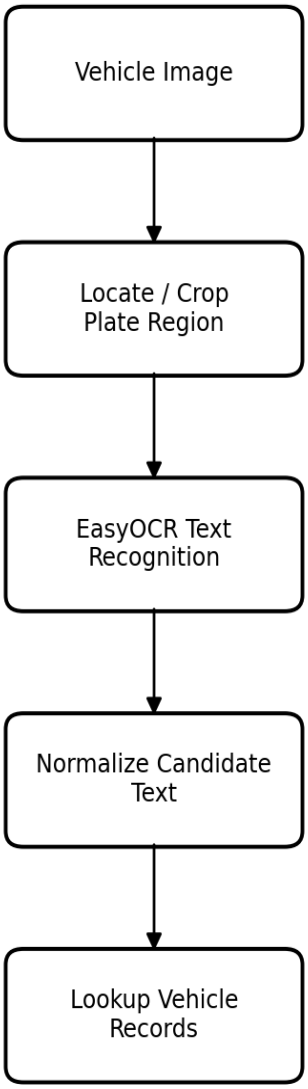
4.3 Automatic Number-Plate Recognition Using EasyOCR

Automatic Number-Plate Recognition (ANPR) is used to obtain candidate text from a vehicle's registration plate. In the proposed system, the available vehicle or plate image is prepared and passed to EasyOCR, which returns text candidates.

The resulting text can be normalized for spacing and formatting and searched against stored vehicle records. Where supported by the implementation, a matching or fuzzy-search mechanism can help identify a possible record. If no reliable match is found, the result should be treated as unmatched rather than automatically assumed to be correct.

Recognition quality depends on plate visibility, image resolution, camera angle, illumination, motion blur, dirt, and partial obstruction. For this reason, extracted text should be considered a candidate until it has been checked against the application's matching rules or reviewed by an administrator.

Figure 4.2 Number-Plate Recognition Workflow



4.4 Random Forest Parking Availability Prediction

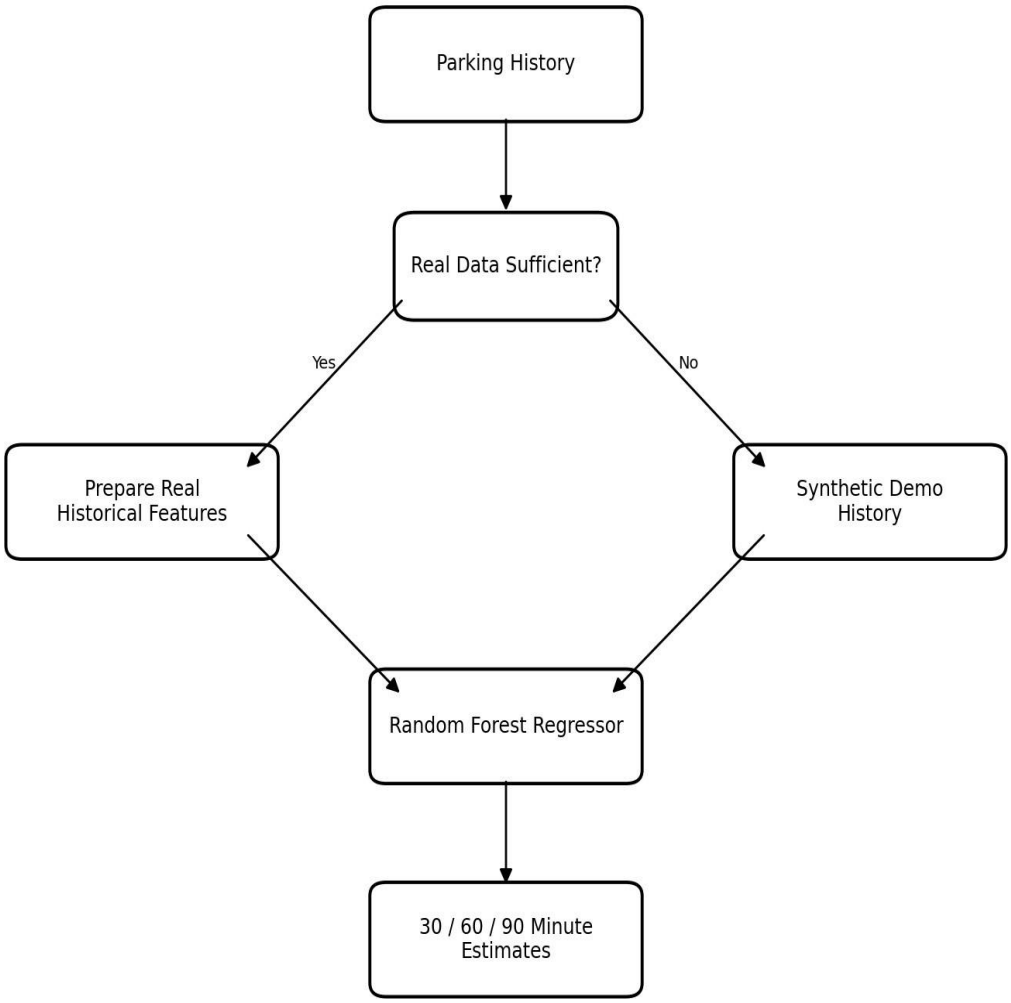
Random Forest regression is used to demonstrate how machine learning can estimate future parking availability. The method combines predictions from multiple decision trees to produce a numerical estimate.

Depending on the available dataset and implemented features, a parking prediction model may use current occupancy, historical availability, time of day, day of the week, or other relevant variables. Only features actually provided to the model should be described as implemented.

In the current prototype, synthetic demonstration history is used because sufficient real historical parking data is unavailable. The resulting estimates for 30-, 60-, and 90-minute horizons demonstrate the prediction workflow; they do not establish real-world forecasting accuracy.

A proper evaluation would require collecting time-stamped observations from actual parking operations, separating training and testing data chronologically, comparing predictions against later observed availability, and calculating suitable measures such as Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE). These measures should be reported only after the corresponding evaluation has been performed.

Figure 4.3 Parking Availability Prediction Workflow



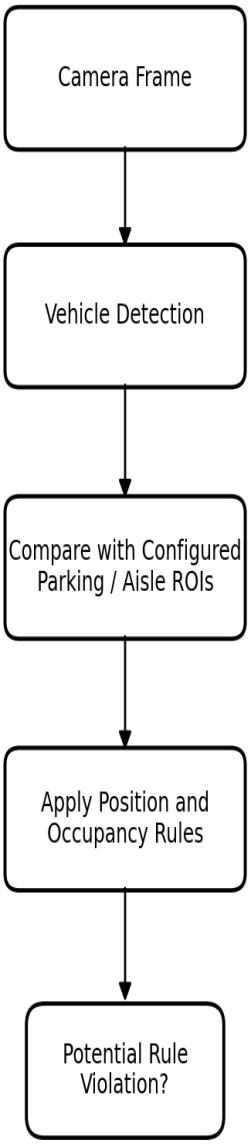
4.5 ROI-Based Rule-Driven Parking Anomaly Detection

A Region of Interest (ROI) is a predefined area within an image representing a parking bay, an aisle, or another relevant region. The prototype uses configured regions and vehicle detections to apply geometric or occupancy-related rules.

For example, a rule may flag a vehicle detection that overlaps an aisle region or does not align with the expected parking-bay region. The system can present such flags for administrator review.

This module is rule-driven rather than a trained anomaly-classification model. A flagged event indicates a potential issue, not definitive proof of incorrect parking. False positives may occur because of inaccurate ROI configuration, perspective distortion, overlapping bounding boxes, or incorrect vehicle detections. The reliability of the checks therefore depends on camera placement, ROI configuration, and the rules implemented in the application.

Figure 4.4 ROI-Based Rule-Driven Anomaly Checking



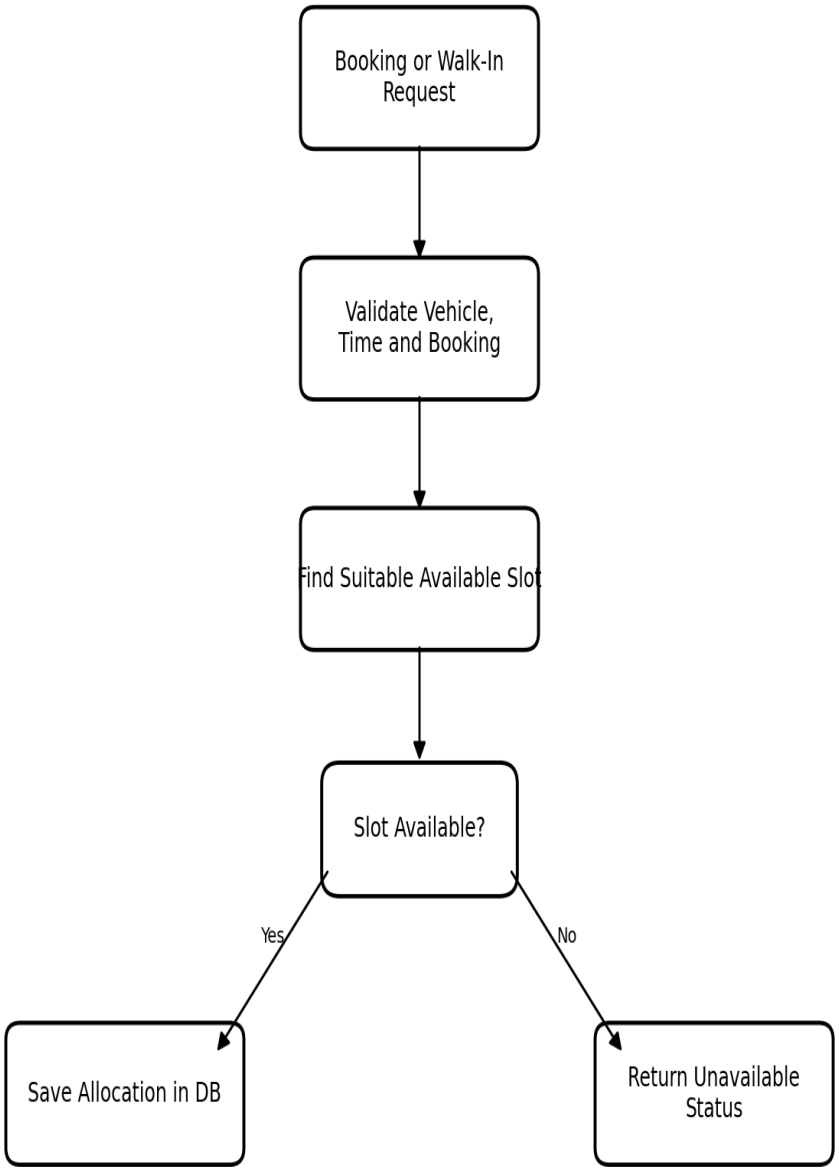
4.6 Dynamic Slot Allocation

Dynamic slot allocation manages requests for parking spaces from users who have booked in advance or arrive without a reservation. The system validates the request, checks the relevant booking and vehicle details, retrieves eligible slots, selects an available slot, and stores the allocation.

The slot state must be updated consistently with the booking or parking-session record. A slot should not be allocated to multiple active requests. Rechecking availability before saving an allocation and using appropriate database transaction handling can help prevent conflicting assignments. Booking expiration and slot release must also follow the rules actually implemented by the application.

The allocation process depends on the accuracy of stored slot states and the completeness of request validation. An unavailable response should be returned when no suitable slot can be assigned.

Figure 4.5 Dynamic Slot Allocation Workflow



4.7 AI Component Summary

Table 4.1. AI and vision component inputs, outputs, and limitations.

Component	Input	Output	Status / limitation
YOLO11n	Image or video frame	Vehicle boxes, classes, confidence values	Pretrained; local performance not yet established
EasyOCR	Plate or vehicle crop	Candidate plate text	Pretrained; affected by image quality
Random Forest	Historical or synthetic features	Estimated availability	Synthetic demo history used when real history is insufficient
ROI rules	Vehicle boxes and configured regions	Potential anomaly flag	Rule-based; not a trained classifier
Slot allocator	Booking/walk-in request and stored slot states	Selected slot or unavailable response	Requires consistency checks and tests

4.8 Summary

This chapter described the AI and analytics components used in the Smart Parking Management System. YOLO11n supports vehicle detection, EasyOCR generates candidate number-plate text, and the Random Forest regressor demonstrates future parking-availability estimation. ROI-based rules identify potential parking anomalies, while the allocation workflow connects parking requests with stored slot information.

The outputs of these components require appropriate validation and integration with database records and application rules. In particular, synthetic-data predictions should not be presented as validated forecasts, and ROI-based flags should not be treated as confirmed violations. These limitations provide a basis for the system design, implementation, and testing discussed in the following chapters.

CHAPTER 5 – SYSTEM DESIGN

5.1 Introduction

System design is an important phase in the development of the AI-Driven Smart Parking Management System. It defines the overall architecture, software modules, database relationships, algorithms, and communication between different components. A well-structured design helps ensure that the system operates efficiently, maintains consistent parking information, and provides a clear interface for users and administrators.

The proposed system integrates computer vision, machine learning, backend processing, database management, and web-based interfaces to simplify parking operations. YOLO11n is used for vehicle detection, while EasyOCR supports number-plate text recognition. A Random Forest regression model estimates future parking availability, and region-of-interest (ROI) rules help identify potential parking anomalies.

The backend manages essential operations such as vehicle records, slot allocation, reservations, parking sessions, and billing. SQLite provides local data persistence, while FastAPI exposes the application functionality through API endpoints.

An important design principle is the separation of AI-based perception from parking business logic. Vehicle detections and recognized number plates are treated as candidate results. The application must validate these results before using them to modify parking records or slot statuses. This approach helps reduce incorrect updates and improves the consistency of the system.

5.2 Overall System Architecture

The overall system architecture describes how the input image, AI processing modules, backend services, database, and user interfaces interact. Each component performs a specific responsibility and exchanges information with the other components through defined workflows.

The system receives images or video frames from the configured camera source. OpenCV supports image processing, and YOLO11n identifies supported vehicle classes. Number-plate imagery is processed using EasyOCR to obtain candidate registration text. The resulting information can be passed to the backend for vehicle lookup and parking operations.

The FastAPI backend acts as the central application layer. It processes requests, validates input data, applies parking rules, and communicates with the SQLite database. The backend also supports booking management, walk-in slot allocation, parking-session tracking, billing calculations, and availability prediction.

The administrator and user interfaces display information returned by the backend. Administrators can access supported management functions, while users can access available parking information and supported booking or parking features.

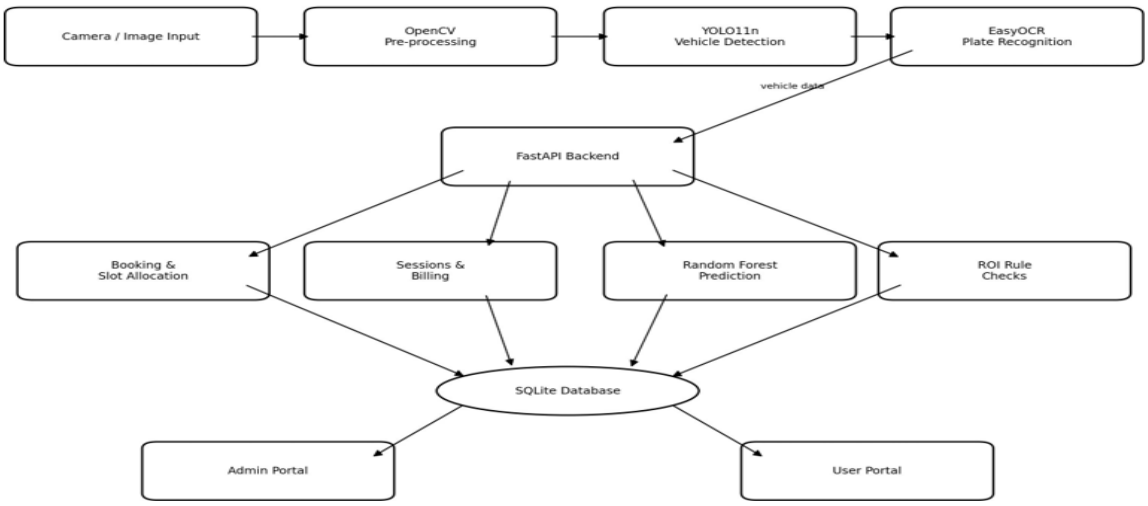


Figure 5.1. Overall system block diagram.

5.3 Module Descriptions

The system is divided into independent but interconnected modules. This modular design makes the application easier to develop, test, maintain, and extend. Each module is responsible for a particular task, and the backend coordinates the flow of information between them.

Table 5.1. Responsibilities of the main software modules.

Module	Responsibility	Key inputs/outputs
Vehicle detection	Run YOLO11n on frames	Input image; vehicle detections
Plate recognition	Run EasyOCR on plate imagery	Plate crop; candidate text
Vehicle records	Store and retrieve vehicle data	Vehicle details and lookup result
Slot management	Maintain slot status and floor/section identity	Slot records and status updates
Booking	Validate and store reservations	Request details and booking state
Walk-in allocation	Allocate a free slot for an arrival	Vehicle/request and assigned slot
Parking session	Track entry, exit, and session state	Session timestamps and slot state
Billing	Compute charges using configured rules	Duration, vehicle type, configured rate
Prediction	Return estimated future availability	Feature data and forecast values
ROI checks	Apply configured geometric rules	Detections and potential anomaly flags
Admin/user interface	Display supported actions and summaries	API responses and user input

5.4 Database Design

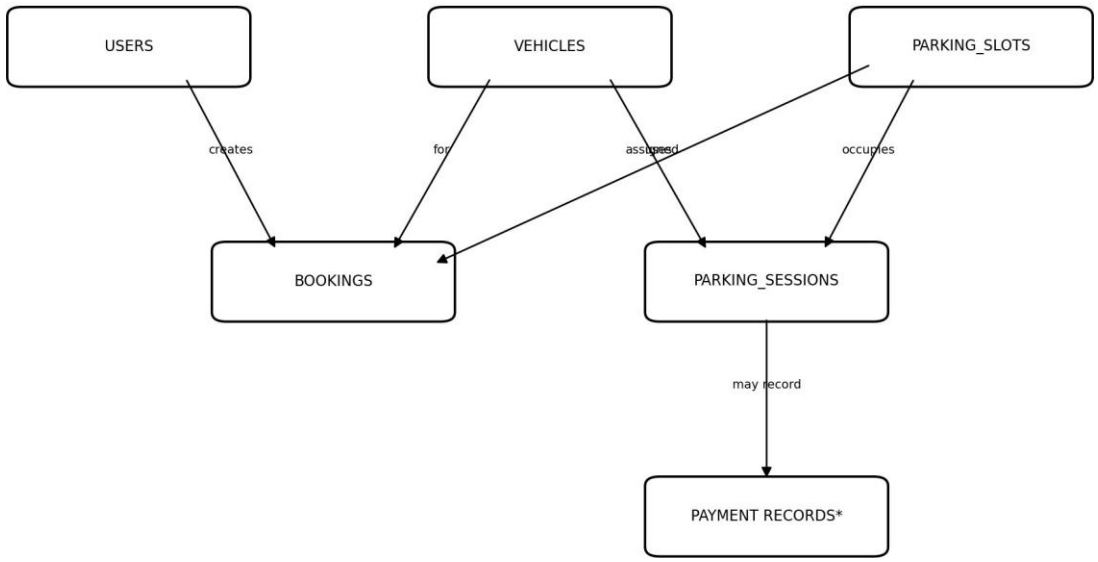
The database stores the information required for the system's parking operations. SQLite is used for local persistence in the prototype. The backend reads and updates records to support vehicle lookup, slot allocation, reservations, parking sessions, and related operations.

Database design is important because multiple modules depend on consistent records. For example, a booking must refer to a valid slot, and a completed parking session must contain sufficient information to calculate the parking charge.The following table presents the conceptual data entities associated with the system. The final field names, primary keys, foreign keys, and relationships must be checked against the actual database schema in the source code

Table 5.2. Conceptual project data entities.

Entity (conceptual)	Possible data fields	Purpose
Users	Username, password hash or credential field, role	Account and access information
Vehicles	Plate number, vehicle category, owner/user reference	Vehicle identification
Parking slots	Slot ID, floor, section, status, slot type	Current slot inventory
Bookings	Booking ID, vehicle/user, slot, start/end time, status	Reservation records
Parking sessions	Session ID, vehicle, slot, entry/exit timestamps, status	Active and completed parking visits
Payments / billing	Amount, rate, payment status, linked session	Billing record if represented separately

Figure 5.2 Conceptual Entity-Relationship View



5.5 Parking-Slot Status Management

Parking-slot status management is essential for preventing incorrect allocations and maintaining reliable parking information. Each slot is represented by a status indicating whether it is available, reserved, occupied, or unavailable, where these states are supported by the implementation.

Table 5.3. Parking-slot status definitions.

Status	Meaning	Typical state transition
Available	Slot is free for allocation	Available → Reserved or Occupied
Reserved	Slot is assigned to a valid booking	Reserved → Occupied, or Available after expiration/ cancellation
Occupied	An active parking session uses the slot	Occupied → Available after session closure
Unavailable (if supported)	Slot is blocked or not usable	Unavailable→ Available after administrator update

5.6 Algorithm Design

The AI-Driven Smart Parking Management System combines computer-vision processing with application-level algorithms. The algorithms described in this section explain the intended processing steps. Exact implementation details, such as confidence thresholds, allocation rules, text-matching thresholds, and billing formulas, should be confirmed against the source code.

5.6.1 Vehicle Detection Algorithm

Objective: To detect supported vehicle classes from an image or video frame.

Input: Image or video frame from the configured source.

Output: Filtered vehicle detections containing bounding boxes, class labels, and confidence values.

Algorithm:

1. Read an image or video frame from the configured input source.
2. Verify that the frame is available for processing.
3. Apply any required image conversion or resizing.
4. Pass the frame to the pretrained YOLO11n detector.
5. Retrieve the predicted bounding boxes, class labels, and confidence values.
6. Filter the detections to retain only the vehicle classes supported by the application.
7. Return the filtered detections to the display or application pipeline.
8. If video processing is active, continue with the next frame.

Result: The application receives candidate vehicle detections for subsequent processing. Detection results may be affected by image quality, lighting, occlusion, and camera position.

5.6.2 Number-Plate Recognition Algorithm

Objective: To obtain candidate vehicle registration text from an image**.**

Input: Vehicle image or number-plate crop.

Output: Candidate plate text and a matching or unmatched result.

Algorithm:

1. Obtain the plate image using the localization method implemented in the application or use a supplied plate crop.
2. Check whether the image is suitable for text recognition.
3. Pass the image to the EasyOCR recognition engine.
4. Retrieve the recognized text candidate.
5. Normalize spacing and character formatting while preserving uncertainty where recognition is unclear.
6. Search the stored vehicle records using the recognized text or the matching method supported by the application.
7. If a suitable record is found, return the candidate match to the application workflow.
8. If no suitable record is found, return an unmatched result for further review.

Result: The application receives candidate registration text that can be associated with a stored vehicle record when the matching rules permit it. OCR output is not guaranteed to identify a vehicle correctly.

5.6.3 Parking-Slot Allocation Algorithm

Objective: To assign an eligible parking slot to a booking or walk-in request.

Input: Booking or walk-in request, vehicle details, time constraints, and current parking-slot records.

Output: Allocation confirmation with slot details, or an unavailable response**.**

Algorithm:

1. Receive the booking or walk-in request.
2. Validate the required vehicle details, time constraints, and booking state.
3. Retrieve candidate parking-slot records from the database.
4. Filter the records according to availability and vehicle compatibility.
5. Select an eligible slot according to the configured allocation policy.
6. Recheck the selected slot's availability before committing the allocation.
7. Save the booking or parking session and update the slot state consistently.
8. Return the allocation confirmation with the assigned slot details.
9. If no eligible slot is available, return an unavailable response without creating an invalid allocation.

Result: The request receives a confirmed allocation when an eligible slot is available. Related database records should remain consistent if a request fails or another request attempts to allocate the same slot**.**

5.6.4 Parking Billing Algorithm

Objective: To calculate the parking charge for a completed parking session**.**

Input: Session entry time, exit time, vehicle category, and configured parking rate**.**

Output: Calculated parking amount and billing or receipt information.

Algorithm:

1. Retrieve the relevant parking-session details.
2. Validate that the session contains valid entry and exit timestamps.
3. Confirm that the exit time is not earlier than the entry time.
4. Calculate the total parking duration.
5. Apply the duration-rounding rule implemented in the application.
6. Retrieve the applicable rate for the vehicle category.
7. Calculate the charge using the configured pricing formula.
8. Store the calculated amount and the corresponding payment status.
9. Return the billing or receipt information.

Result: The system produces a parking charge based on session duration and the configured rate. The exact amount depends on the pricing and rounding rules implemented in the application. Online payment processing is not assumed unless a payment gateway has been integrated and tested.

5.6.5 Parking Availability Prediction Algorithm

Objective: To estimate the number of available parking slots at supported future time horizons**.**

Input: Historical parking features, or synthetic demonstration data when real historical data is insufficient.

Output: Estimated parking availability for supported future intervals.

Algorithm:

1. Check whether sufficient historical parking data is available.
2. Load suitable historical features when available.
3. If real historical data is insufficient, use synthetic demonstration data and identify the output accordingly.
4. Prepare the feature and target arrays according to the implemented data-processing pipeline.
5. Fit or load the Random Forest regressor according to the application's implementation.
6. Prepare the input features for the required prediction horizons.
7. Generate availability estimates for the supported future time intervals.
8. Display the estimates in the application interface.
9. Indicate when predictions are based on synthetic demonstration data and have not been validated against real parking observations.

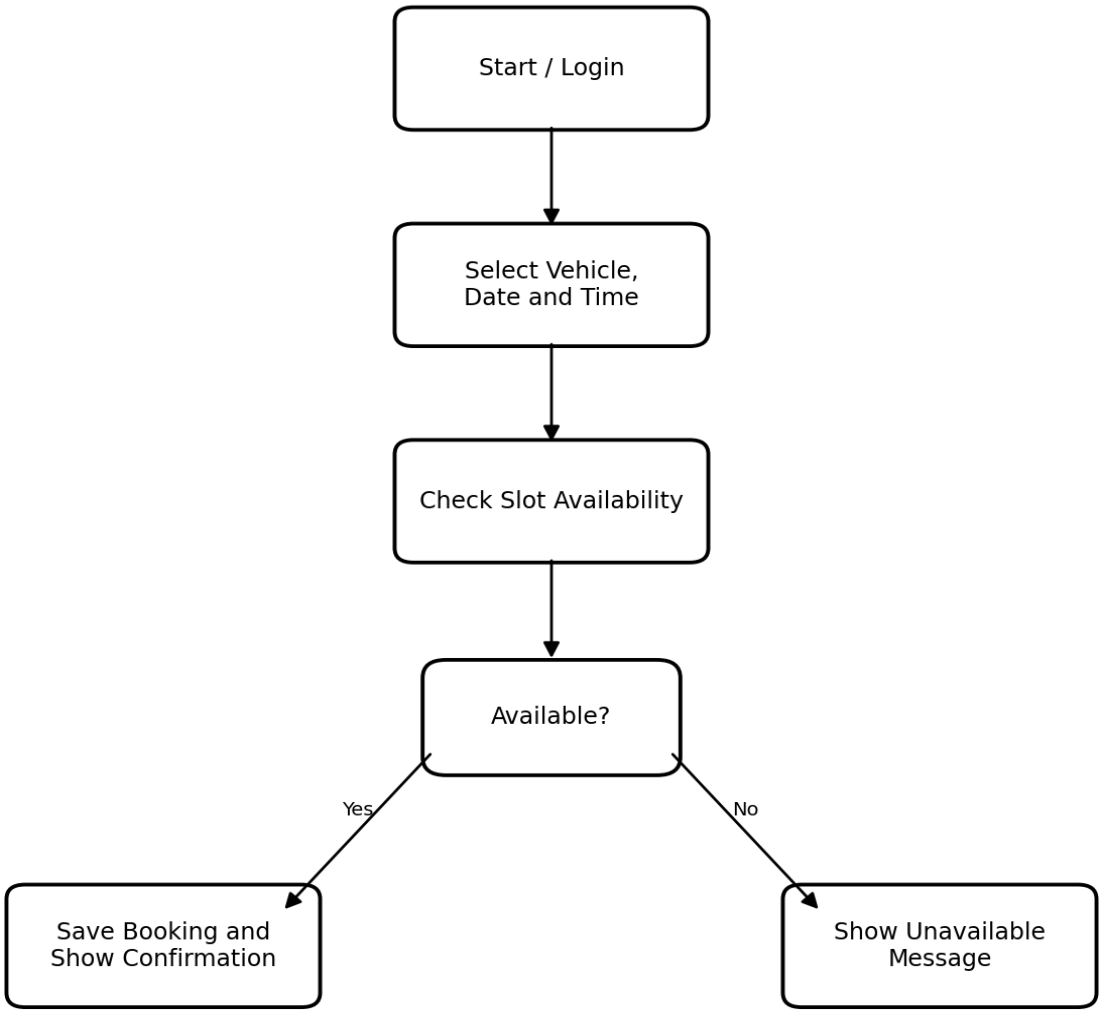
Result: The system displays estimated future availability to support demonstration and planning. Since the current prototype uses synthetic data when real history is insufficient, predictive accuracy and real-world reliability have not yet been established**.**

5.7 Booking Activity Diagram

The booking activity diagram represents the sequence of actions performed when a user requests a parking slot. The process begins when the application receives a booking or walk-in request. It validates the request, checks for an eligible slot, and then either returns an error or unavailable response or proceeds with the allocation.

When a suitable slot is found, its availability should be rechecked before the application saves the booking or session. After the relevant records are updated, the application returns an allocation confirmation to the requester.

Figure 5.3 Booking Activity Diagram



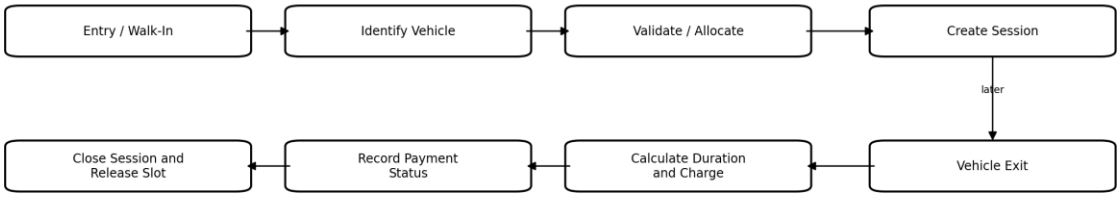
5.8 Parking Session and Billing Workflow

The parking-session and billing workflow describes how the application manages a parking visit from the initial session request through the calculation of the final charge. When a valid entry request is received, the backend checks the relevant vehicle and slot information and records or updates the parking session according to the implemented workflow.

At exit, the application retrieves the session details and validates the timestamps. It calculates the duration, applies the configured rate for the vehicle category, and generates the billing information. The resulting amount and payment status are then stored according to the database design.

The workflow depends on valid session records and consistent slot updates. If an entry or exit timestamp is missing or invalid, the application should handle the problem rather than calculate an unreliable charge.

Figure 5.4 Parking Session and Billing Sequence



5.9 Security and Data Integrity

Security and data integrity are important for protecting vehicle information, user accounts, and parking records. The prototype should validate incoming API requests and handle missing, malformed, or inconsistent input fields before processing them.

Administrative operations should be restricted to authorized users. Passwords should be stored using an appropriate password-hashing method rather than as plaintext. Vehicle registration numbers and captured images may contain personally identifying information, so access should be controlled and stored information retained only for as long as necessary.

Database consistency is also essential. Booking creation, slot allocation, session closure, and billing operations should maintain valid relationships between the associated records. Failed operations should not leave the system with conflicting slot statuses or incomplete session information.

Before deployment in a real parking facility, additional review is required for authentication, authorization, logging, backup procedures, error handling, concurrent requests, and database transaction behavior. These safeguards should be tested rather than assumed to be fully implemented.

CHAPTER 6 – IMPLEMENTATION

6.1 Introduction

The implementation phase describes how the AI-Driven Smart Parking Management System is developed by integrating computer vision, backend services, database management, parking operations, and web-based interfaces. The system is organized into separate modules to simplify development, testing, and maintenance.

The FastAPI application is implemented in backend/main.py, while database operations and business logic are managed by modules within the backend/ directory. Camera-related workflows are implemented in the camera/ directory, and the Random Forest prediction model is maintained in ml/parking_prediction.py. The administrator and user interfaces are available through the frontend/ directory.

The application stores its records in database/parking.db using SQLite. The web server and camera-monitoring processes operate separately and communicate through the backend API. This structure allows the system to manage parking operations while processing vehicle images and generating availability estimates.

6.2 Development Environment

The system is developed using Python and supporting libraries for web development, computer vision, machine learning, and database management. Each technology performs a specific role in the prototype.

Table 6.1. Technology stack used by the prototype.

Technology	Role in the prototype
Python	Main programming language
FastAPI	Backend API framework
SQLite	Local database
OpenCV	Image/video handling
Ultralytics YOLO11n	Pretrained vehicle detector
EasyOCR	Pretrained OCR engine
scikit-learn	Random Forest regression
Uvicorn (if used)	ASGI server for FastAPI
Browser UI (if present)	User/admin interaction

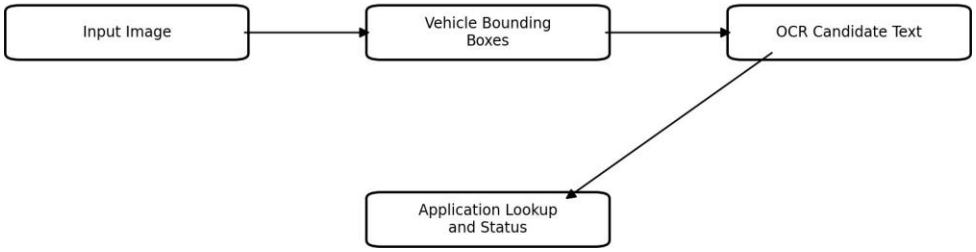
6.3 Camera and Vehicle Detection Implementation

The camera module processes images or video frames to identify vehicles and support parking monitoring. The file camera/plate_camera.py loads the pretrained YOLO11n model and processes frames from the configured camera index.

The separate file camera/parking_detection.py runs a YOLO-based vehicle detection process and checks configured parking-bay and aisle regions. Based on the configured rules, the process sends occupancy changes and observations to the backend through API requests.

The parking-bay rectangles and aisle region are manually configured examples. They must be calibrated to match the camera position and actual parking layout. Vehicle detection alone does not establish a verified parking-bay state; the application must consider the configured regions and existing parking records before applying updates**.**

Figure 6.1 Illustrative Vehicle Detection Pipeline



6.4 Number-Plate Recognition Implementation

The number-plate recognition module is implemented in camera/plate_camera.py. It uses YOLO-based vehicle detection together with EasyOCR to produce candidate registration text from the available image regions.

After a candidate plate is obtained, the backend normalizes the recognized text and passes it to the relevant vehicle-entry or vehicle-exit workflow. The resulting text can be used to search for a corresponding vehicle record.

During the demonstration, four cars were detected. However, the displayed OCR candidates had low confidence and were not verified as readable registration numbers. Therefore, the implementation demonstrates the integration of candidate recognition and backend processing, but it does not establish number-plate recognition accuracy.

The recognition results may be affected by image quality, lighting conditions, camera angle, motion blur, and partially obstructed plates. Further testing with representative vehicle images is required to evaluate the reliability of the module**.**

6.5 FastAPI Backend and SQLite Database

The backend acts as the central application layer of the smart parking system. The file backend/main.py defines the FastAPI routes and imports the modules responsible for slot management, booking, vehicle entry and exit, billing, analytics, monitoring, and availability prediction.

During application startup, the backend calls create_tables(), creates sample parking slots, and trains the availability prediction model. The database path is configured in backend/database.py as database/parking.db.

The backend receives requests from the camera processes and user interfaces, validates the required information, executes the relevant business logic, and interacts with the database. The database stores the parking records used by the different modules.

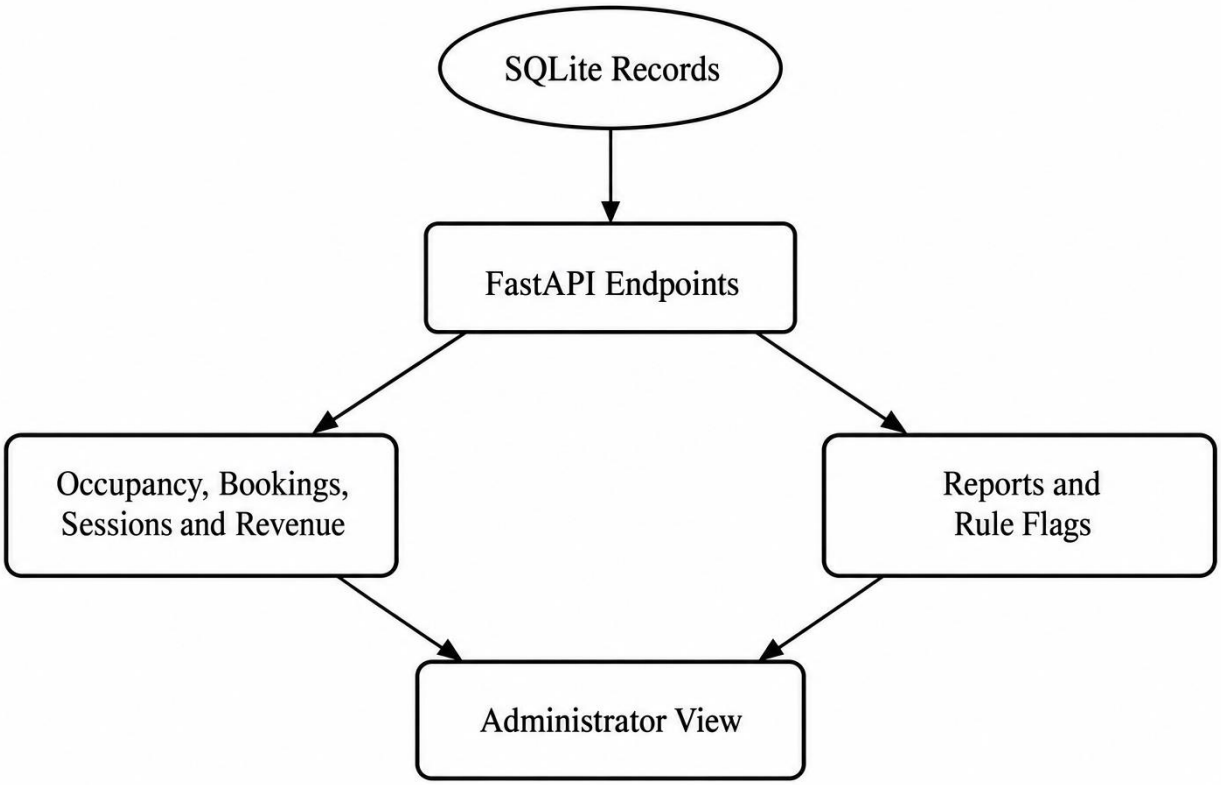
The implemented database tables and API routes are documented in Appendix C. The exact schema and route behavior should be verified against the project source code before final submission.

6.6 Administrator Dashboard Implementation

The administrator dashboard is accessible through the /dashboard route. It provides an overview of parking operations and presents information about slot availability, bookings, vehicle records, parking sessions, billing, predictions, and alerts.

The dashboard obtains information from the backend and displays the records available in the application database. This interface helps administrators observe the current stored parking state and review the information generated by the system.

Figure 6.2 Administrator Dashboard Information Flow



6.7 Parking-Slot Availability View

The parking-slot availability view organizes 40 sample parking slots by floor. Each slot is displayed using its stored status, such as Available, Occupied, or Reserved.

The separate camera/parking_detection.py process can update occupancy information for unassigned parking bays based on camera frames. However, confirmed bookings and active or pending sessions are protected from vision-based updates to prevent the camera process from incorrectly overriding existing parking records.

The dashboard provides a visual representation of the stored slot information and helps users or administrators understand the distribution of available and occupied spaces.

Figure 6.3 Parking Slot Status View

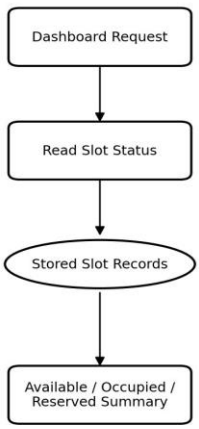
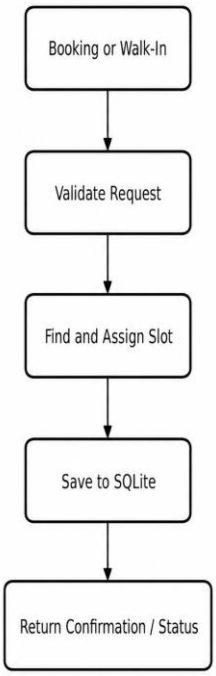


Figure 6.4 Booking and Walk-In Allocation



6.8 Booking and Walk-In Allocation

The user portal submits parking reservations through the POST/ api/ book endpoint. When a booking is confirmed, the selected parking bay is held for that reservation according to the application's booking logic.

When a vehicle arrives, the vehicle entry() function checks the recognized plate and associated booking information. If a matching confirmed booking is found, the vehicle is assigned its reserved bay. If no confirmed booking applies, the system can allocate an available walk-in bay according to the implemented rules.

The booking manager also provides a route for cancelling expired bookings. This helps prevent expired reservations from continuing to occupy parking spaces unnecessarily.

6.9 Parking Sessions and Billing

The parking-session module records information about a vehicle's parking visit, including the assigned slot, entry time, and session status. When a vehicle exits, the system creates a pending bill based on the recorded parking duration and the configured vehicle category.

The billing logic is implemented in backend/billing_manager.py. The system charges for at least one hour, rounds partial hours upward, and applies the configured base rate for the first two billable hours. For subsequent billable hours, the rate is 1.5 times the base rate.

6.10 Prediction and Anomaly Checks

ml/parking_prediction.py fits a RandomForestRegressor at application startup and returns estimates for 30-, 60-, and 90-minute horizons. The training routine can generate synthetic parking history when the

database lacks enough observations, so predictions demonstrate integration rather than validated forecast quality. Record-based anomaly checks identify no-show bookings and duplicate active sessions. The separate vision monitor can submit slot mismatch, double-parking, improper-parking, and aisle-obstruction observations.

6.11 API Endpoint Documentation

Table 6.2 lists principal application routes from backend/main.py. Appendix C provides the complete route inventory. Authentication routes exist, but the authentication dependency is not consistently applied to other API routes.

Table 6.2. Principal FastAPI endpoints used by the portals.

HTTP method	Endpoint	Purpose	Verification
GET	/api/slots	List slot states	Backend route
GET	/api/slots/{slot_id}	Retrieve one slot	Backend route
GET	/api/slots/{slot_id}/details	Retrieve slot details	Backend route
PUT	/api/slots/{slot_id}/status	Update slot status	Backend route
POST	/api/register	Register a vehicle	Backend route
POST	/api/book	Create a booking	Backend route
GET	/api/bookings	List bookings	Backend route
POST	/api/entry	Record vehicle entry	Backend route
POST	/api/exit	Record vehicle exit and bill	Backend route
POST	/api/camera/start	Start entry/exit camera process	Backend route
GET	/api/camera/latest	Read latest camera event	Backend route
GET	/api/prediction	Predict availability	Backend route
POST	/api/payments/{session_id}/pay	Confirm demo payment	Backend route
GET	/api/anomalies	List anomaly and vision alerts	Backend route
POST	/api/vision/observations	Record a vision observation	Backend route
POST	/api/auth/login	login	Auth route
POST	/api/auth/user/login	User login	Auth route
GET	/api/auth/verify	Verify token	Auth route

6.12 Testing and Validation Approach

Testing was performed through local demonstrations of the web interfaces, camera-processing workflows, and parking-management functions. Appendix B contains screenshots and outputs captured on 14–15 October 2026 (UTC+05:30).

The screenshots demonstrate selected interactions within an isolated demonstration environment. They provide evidence of the displayed interface and recorded sample operations, but they do not establish model accuracy, production reliability, or performance under real parking-facility conditions.

6.13 Implementation Limitations

The current prototype has several limitations that must be considered when interpreting its results.

First, the availability prediction module can use synthetic parking-history data when real observations are insufficient. Consequently, its predictions have not been validated as accurate real-world forecasts.

Second, YOLO11n and EasyOCR are pretrained models. No locally measured detection accuracy or number-plate recognition accuracy is reported in this implementation. The camera regions are manually configured and require calibration for different camera positions and parking layouts.

Third, the prototype does not include physical occupancy sensors, an automated parking barrier, or an integrated real payment gateway. Payment confirmation is a demonstration function.

Security also requires improvement. Demonstration passwords are stored as plaintext, and authentication dependencies are not consistently applied to other API routes. These limitations create risks if the application is exposed to unauthorized users.

Further validation is needed for concurrent requests, database consistency, privacy protection, backup and recovery, error handling, and deployment under real operating conditions. These improvements are necessary before the prototype can be considered suitable for production use.

CHAPTER 7 – RESULTS AND DISCUSSION

7.1 Introduction

This chapter presents the observed results of the AI-Driven Smart Parking Management System using an isolated local demonstration environment. The evaluation focuses on the functionality demonstrated by the administrator dashboard, parking-slot map, vehicle detection, number-plate recognition, booking management, walk-in allocation, parking-session billing, availability prediction, and anomaly-monitoring features.

The available evidence consists of screenshots and recorded application outputs from selected operations. These examples demonstrate the integration of the software modules and the processing of sample parking records. However, controlled model-accuracy measurements and comprehensive performance evaluations have not been conducted.

Therefore, the results presented in this chapter should be interpreted as evidence of prototype functionality rather than proof of production readiness or validated AI performance.

7.2 Prototype Overview

The prototype integrates a FastAPI backend, an SQLite database, user and administrator portal pages, and separate camera-processing scripts. The backend manages parking records and application operations, while the camera modules provide vehicle detections and candidate number-plate text.

The administrator dashboard displayed 40 total parking slots at the time of the recorded snapshot. Of these, 18 were available, 16 were occupied, and 6 were reserved. These values represent the stored state of the demonstration database at the time of capture.

Table 7.1. Demonstration values recorded in the evidence screenshots.

Metric	Demonstration value	Interpretation
Total parking slots	40	Figure B.1
Available slots	18	Figure B.1
Occupied slots	16	Figure B.1
Reserved slots	6	Figure B.1
Walk-in session	WALK516 active in B1	Figure B.5
Walk-in session	WALK514 awaiting payment in D10	Figure B.5
Pending bill	₹10; one billable hour	Figure B.6
Prediction at 30 minutes	26 available	Figure B.7
Prediction at 60 minutes	28 available	Figure B.7
Prediction at 90 minutes	30 available	Figure B.7

The screenshots record separate actions and timestamps, so their counts should not be merged into one simultaneous state. The 30-, 60-, and 90-minute availability values are model outputs from a demo run and are not evidence of forecast accuracy.

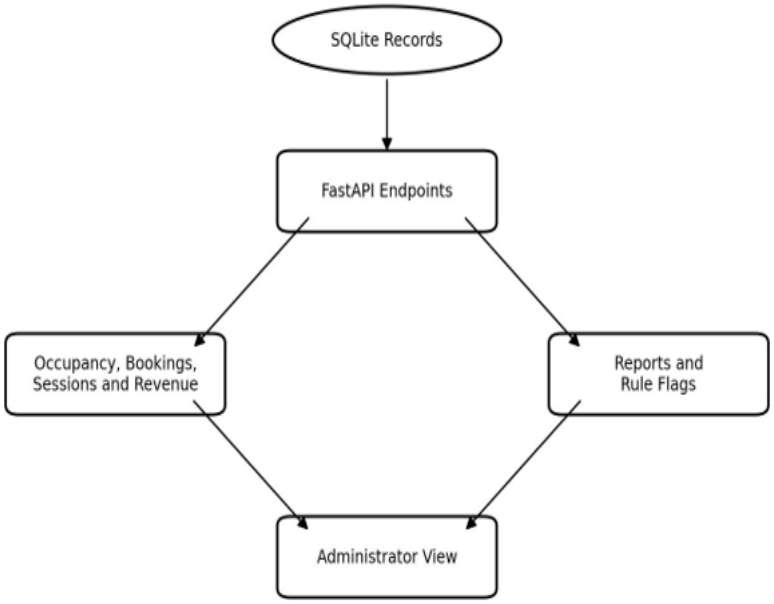


Figure 7.1. Dashboard data flow; the interface snapshot appears in Figure B.1.

7.3 Vehicle Detection and Number-Plate Recognition

The vehicle detection demonstration is presented in Figure B.3. The YOLO11n model identified four cars in the supplied parking image, demonstrating that the pretrained detector could produce candidate vehicle detections in the tested example.

The number-plate recognition component used EasyOCR to produce candidate text. However, the displayed OCR candidates had low confidence and were not verified as readable vehicle registration numbers. Therefore, the available evidence demonstrates the integration of vehicle detection and OCR processing, but it does not establish reliable number-plate recognition.

Quantitative evaluation would require a representative set of labelled vehicle images, annotated bounding boxes, verified registration strings, and appropriate evaluation metrics. Since such an evaluation is not reported, no detection or OCR accuracy value is claimed.

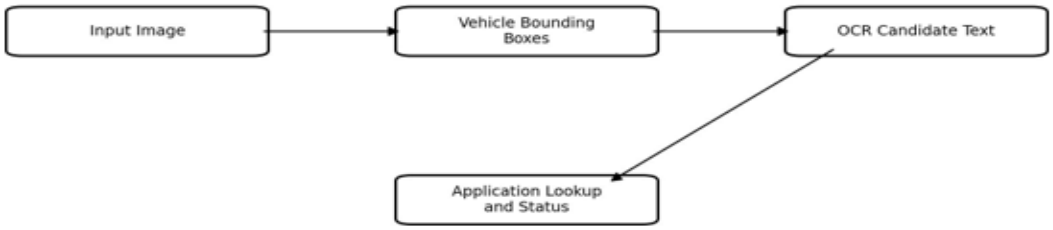


Figure 7.2. Vehicle identification workflow; observed output appears in Figure B.3.

7.4 Booking and Session Management

The booking and session-management features were demonstrated using sample records from the local application instance.

Figure B.4 records a successful booking for vehicle TEST1242, assigned to parking slot A4. Figure B.5 presents two walk-in session examples: WALK516 is shown as active in slot B1, while WALK514 is shown awaiting payment after exiting from slot D10.

These examples demonstrate selected booking, allocation, and session-record workflows. They also show how the application represents the status of parking visits within the demonstration environment.

However, the available screenshots do not establish correct behavior under simultaneous booking requests, conflicting allocations, or all expired-reservation scenarios. These cases require additional functional and concurrency testing.

.

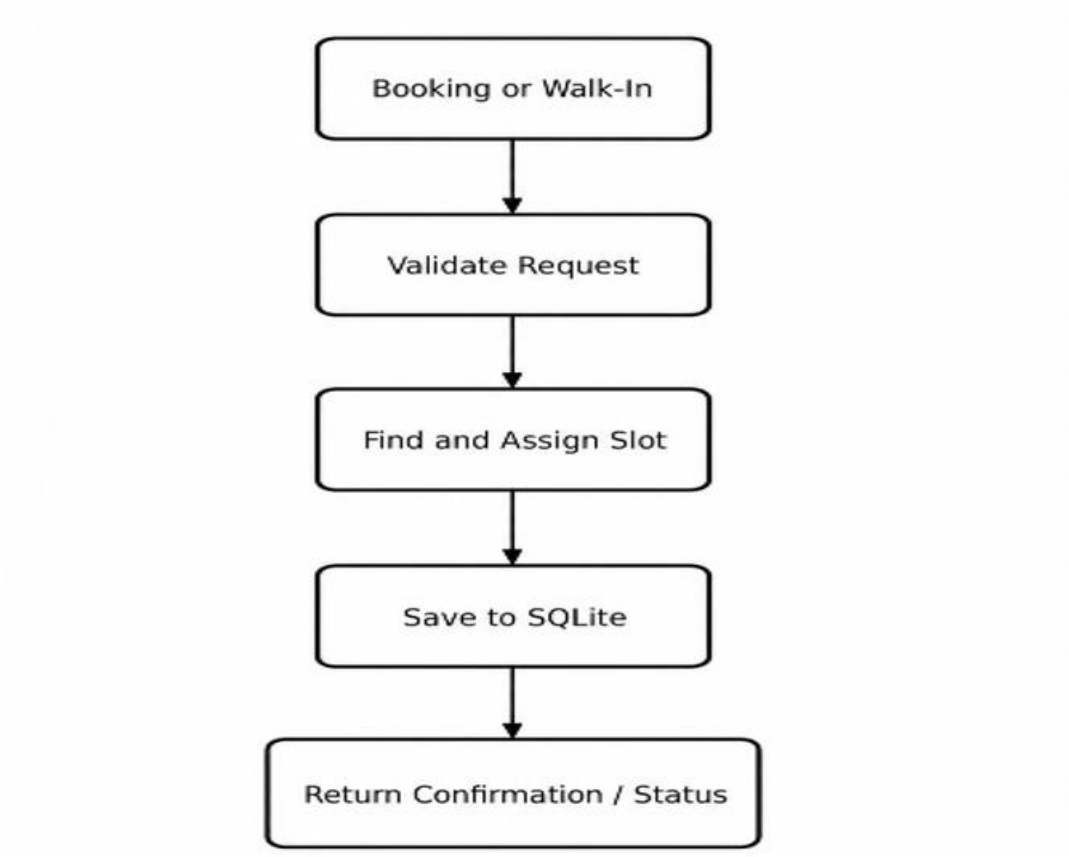


Figure 7.3. Booking and session workflow; observed examples appear in Figures B.4 and B.5.

7.5 Parking Availability Prediction

The availability prediction module uses a Random Forest regressor to estimate the number of available parking slots at future time intervals. The demonstration interface displayed the following outputs:

- 26 available slots after 30 minutes.
- 28 available slots after 60 minutes.
- 30 available slots after 90 minutes.

These values are outputs from a demonstration run. The prediction routine can use synthetic parking-history data when sufficient real historical observations are unavailable. Consequently, the displayed estimates should not be interpreted as measured forecast results or evidence that future parking availability can be predicted accurately.

A future evaluation should collect timestamped occupancy records over a representative period. Predictions should then be compared with actual parking availability observed at the corresponding future times. Suitable error metrics can be reported after this evaluation has been performed.

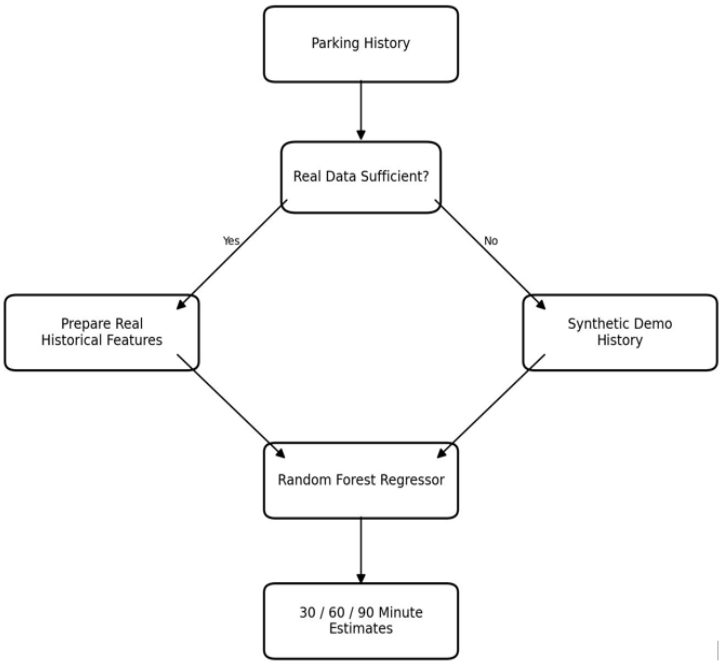


Figure 7.4. Prediction workflow; demo cards appear in Figure B.7.

7.6 Anomaly Detection

The prototype includes record-based checks for identifying no-show bookings and duplicate active parking sessions. In addition, the separate vision-monitoring process can submit observations relating to slot mismatches, multiple vehicles within a parking bay, improper parking, and aisle obstruction.

The record-based checks operate on stored application data, while vision-based observations depend on configured camera regions and the conditions defined by the monitoring process.

Figure B.8 presents an API-submitted test observation. This demonstrates that an observation can be submitted to the application, but it does not establish that the live camera automatically detected the reported condition. The effectiveness of the anomaly-monitoring features therefore requires further validation using labelled observations and representative camera footage.

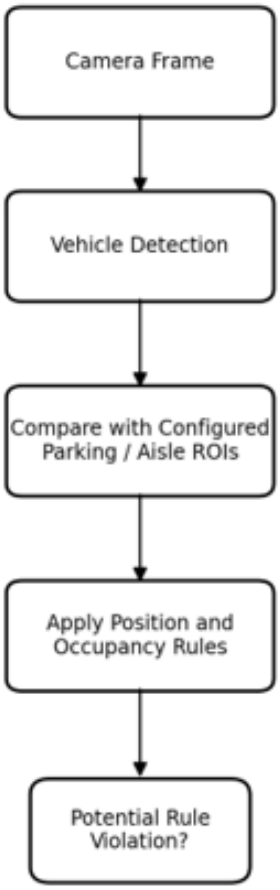


Figure 7.5. ROI-based anomaly-checking workflow; the API-submitted example appears in Figure B.8.

7.7 Billing and Parking Sessions

The billing demonstration is illustrated in Figure B.6. The record for WALK514 displays a parking duration of one minute, an amount of ₹10, a Pending payment status, and an Awaiting Payment state.

The configured base parking rate for a bike is ₹10 per hour. The billing logic charges for at least one hour and rounds partial hours upward. As a result, a one-minute session is charged for one billable hour under the configured rules.

The parking bay remains assigned until payment confirmation. The payment-confirmation interface is a demonstration action, and no external payment provider is integrated with the prototype.

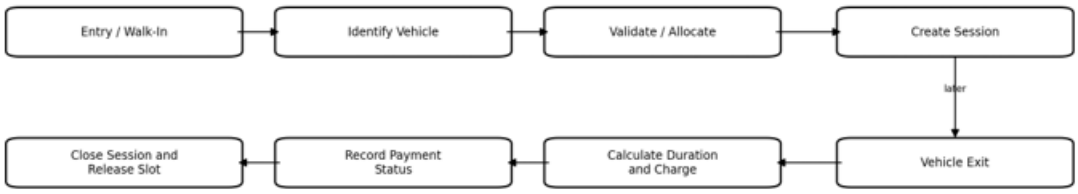


Figure 7.6. Parking-session and billing workflow; the pending-bill example appears in Figure B.6.

7.8 Functional Test Plan

The functional test plan identifies the principal features of the prototype, their intended behaviour, and the evidence available from the demonstration environment. The evidence status distinguishes directly observed examples from cases that remain pending verification.

Table 7.2. Functional test plan and evidence status.

Test ID	Feature	Procedure	Expected result	Evidence status
TC-01	Slot API	Request stored slot records	Returns current stored slot states	Dashboard shows 40 slots; direct API contract not separately captured.
TC-02	Vehicle detection	Run Representative clear/occluded images	Returns candidate vehicle detections	Four cars shown in sample image; no accuracy metric reported.
TC-03	Plate OCR	Test clear and difficult plate images	Returns candidate text or unmatched result	OCR candidates were low-confidence and not validated as plates.
TC-04	Booking	Request a free slot	Booking saved and slot state updated	A4 booking for TEST1242 confirmed in Figure B.4.
TC-05	Unavailable slot	Request an unavailable slot	Request rejected or alternatives returned	Pending; no evidence included.
TC-06	Expiration	Process an expired booking	Slot released according to configured rules	Pending; no evidence included.
TC-07	Walk-in	Create a walk-in allocation	Suitable slot and session recorded	WALK516 active in B1; WALK514 awaiting payment in D10, Figure B.5.
TC-08	Billing	Calculate a test session bill	Amount matches configured rate and duration	WALK514 shows one minute and ₹10 pending, Figure B.6.
TC-09	Prediction	Run prediction endpoint/module	Returns estimates for supported horizons	Demo UI shows 26/28/30 available at 30/60/90 minutes, Figure B.7.

TC-10	Vision observation	Submit slot/ROI observation	Potential alert recorded and displayed	API-supplied alerts in Figure B.8; live camera detection not demonstrated.
TC-11	Persistence	Restart and read records	Persisted data remains available	Pending:no persistence-restart evidence included.
TC-12	Invalid input	Submit malformed fields	Validation error without corrupting records	Pending; no evidence included.

Appendix B contains the available demonstration evidence. Table 7.2 marks only behaviors directly shown in screenshots as observed; unrepresented edge cases remain pending. The full automated test suite was not rerun during this report update.

7.9 Discussion

The demonstration shows that the prototype integrates several important components of a smart parking management system. These include vehicle detection, candidate number-plate recognition, parking-slot management, booking, walk-in allocation, parking-session tracking, billing, availability prediction, and anomaly-observation handling.

One of the principal strengths of the prototype is its modular architecture. Computer-vision processing is separated from backend business logic, while FastAPI and SQLite support the exchange and storage of application information. The administrator dashboard provides a central interface for viewing stored parking information and selected analytics outputs.

The demonstration also identifies important limitations. Vehicle detection and OCR processing have not been evaluated using quantitative accuracy metrics. Parking availability predictions may rely on synthetic training data, and the anomaly-monitoring examples do not establish reliable live-camera detection. In addition, the current evidence does not cover all edge cases, concurrent requests, or production operating conditions.

These findings indicate that successful integration of software modules is an important development milestone, but it is not sufficient to establish production reliability or AI model performance. Additional testing with real parking data, representative camera images, and realistic operating conditions is necessary before the system can be deployed in an actual parking facility.

.

7.10 Limitations Observed

The demonstration identified several limitations that may affect the reliability and generalizability of the prototype. These limitations and their recommended mitigation strategies are summarized below.

Table 7.3. Prototype limitations and mitigations.

Limitation	Potential impact	Recommend ed mitigation
Synthetic prediction history	Forecasts may not match real parking patterns	Collect real time-stamped occupancy data and evaluate chronologically

Pretrained detection/OCR	Local performance may vary	Build a representative labelled test set and report metrics
ROI-based rules	False alarms or missed violations	Calibrate ROIs and compare with human-labelled cases
Camera quality	Blur, glare, and occlusion reduce recognition	Improve placement/lighting and handle uncertain results
No physical sensors	Slot state may depend on software records	Integrate and validate sensors as future work
Prototype database	Concurrent changes may cause inconsistent allocation	Test transactions and concurrent requests

CHAPTER 8 – CONCLUSION AND FUTURE ENHANCEMENT

8.1 Project Summary

The AI-Driven Smart Parking Management System is a software prototype developed to improve parking management through computer vision, backend processing, and predictive analytics. The system integrates pretrained YOLO11n for vehicle detection, pretrained EasyOCR for number-plate text recognition, a FastAPI backend, and SQLite database storage.

The prototype supports parking-slot management, booking and walk-in allocation, parking-session tracking, billing, and administrator dashboard functionality. It also incorporates a Random Forest regression model to demonstrate parking availability prediction and region-of-interest (ROI) rules to identify potential parking anomalies.

The project combines these components within a modular software architecture. The resulting prototype provides a foundation for further development and evaluation of an integrated smart parking management solution.

8.2 Key Achievements

The major achievements of the project are summarized below:

1. Modular system architecture: Established an architecture connecting computer-vision processing, backend application logic, persistent storage, and dashboard interfaces.

2. Vehicle detection: Integrated the pretrained YOLO11n model into the vehicle-processing workflow.

3. Number-plate recognition: Integrated EasyOCR to generate candidate registration text from suitable vehicle imagery.

4. Parking management: Developed software-side functions for slot management, booking, walk-in allocation, and parking-session tracking.

5. Billing functionality: Implemented parking-charge calculation based on configured vehicle categories, hourly rates, and duration-rounding rules.

6. Availability prediction: Integrated a Random Forest prediction demonstration with disclosure of the use of synthetic training history when real historical data is insufficient.

7. Anomaly monitoring: Defined ROI-based rules and record-based checks for identifying potential parking-related anomalies.

8. Testing documentation: Prepared a functional test plan that distinguishes observed demonstration results from features and scenarios requiring further verification.

These achievements demonstrate the integration of the major software components. However, they do not establish validated AI accuracy or production-level reliability.

8.3 Conclusion

The AI-Driven Smart Parking Management System demonstrates the feasibility of combining computer vision and application-level parking management within a single software prototype. By integrating vehicle detection, number-plate text recognition, parking-slot management, booking, session tracking, billing, and demonstration-level analytics, the project provides a foundation for improving the organization and monitoring of parking operations.

The modular architecture allows the different components to communicate through the backend while maintaining parking records in the SQLite database. The administrator dashboard provides a centralized interface for viewing parking information and selected analytical outputs. The availability prediction module and ROI-based checks further demonstrate how machine learning and rule-based processing can be incorporated into the system.

Nevertheless, the current prototype has important limitations. Vehicle detection and number-plate recognition have not been evaluated using reported local accuracy metrics. Availability predictions may rely on synthetic data, and the reliability of the anomaly-monitoring rules has not been established through comprehensive real-world testing. The system also lacks physical occupancy sensors, automated parking barriers, and an integrated real payment gateway.

In conclusion, the project achieves its primary goal of demonstrating an integrated software approach to smart parking management. Further development, systematic testing, and validation using real parking observations are necessary before the system can be considered suitable for operational deployment.

8.4 Future Enhancements

The following enhancements can improve the reliability, security, and practical applicability of the proposed system**.**

8.4.1 Real-Time Parking Data Collection

Future development should focus on collecting real parking-occupancy histories with appropriate consent and privacy safeguards. These records can be used to evaluate availability predictions against actual observations. Suitable forecasting metrics should be calculated using chronological evaluation procedures.

8.4.2 Improved Vehicle and Number-Plate Recognition

A representative local dataset should be prepared to evaluate vehicle detection and number-plate recognition under different lighting conditions, camera angles, distances, and levels of occlusion. Recognition workflows can be improved through image-quality checks, confidence thresholds, and human review of uncertain results.

8.4.3 Integration of Physical Parking Sensors

Physical occupancy sensors may be integrated to provide additional evidence about whether individual parking bays are occupied. Automated barriers could also be considered after the hardware design, control logic, and safety requirements have been validated. These additions would require further hardware and software development.

8.4.4 Enhanced Security and Data Protection

The application should implement consistent authentication and authorization across protected API endpoints. Passwords should be stored using secure password-hashing methods. Additional improvements should include access logging, backup and recovery procedures, privacy controls, and secure handling of vehicle identifiers and captured images.

8.4.5 Improved Slot Allocation and Database Consistency

Future versions should be tested under simultaneous booking and allocation requests. Database transactions and appropriate concurrency controls should be used to prevent conflicting allocations and inconsistent session records. Expired bookings, failed requests, and interrupted sessions should also be tested systematically.

8.4.6 Payment Gateway and Notification Integration

A secure payment gateway may be integrated if real online payments are required. Automated notifications could also inform users about booking confirmations, reservation expiration, and parking-session status. These features should be implemented with appropriate security and error-handling mechanisms.

8.4.7 Performance Optimization and Deployment

The system could be deployed on suitable server or edge-computing hardware after evaluating inference latency, memory usage, processing requirements, and network conditions. Performance testing would help determine whether the system can meet the response-time requirements of an operational parking facility.

8.4.8 Improved Anomaly Monitoring

The ROI-based anomaly rules should be evaluated against labelled examples of slot mismatches, improper parking, multiple vehicles within a bay, and aisle obstruction. Camera-region calibration and rule refinement may reduce false alarms and missed detections. Any improvement should be assessed using documented test results.

8.5 Research Contributions

The primary contribution of this project is the development of an integrated software prototype that combines vehicle detection, number-plate text recognition, parking management, and demonstration-level analytics within a common application architecture.

The project documents how pretrained computer-vision models can be connected to backend services and database-driven parking workflows. It also demonstrates the integration of Random Forest-based availability prediction and rule-based anomaly checks.

The contribution is primarily architectural and implementation-oriented. The project does not claim to introduce a novel trained vehicle-detection architecture or a newly validated forecasting method. Instead, it provides a foundation for future implementation, testing, and data-driven evaluation of smart parking management systems.

8.6 Social and Environmental Impact

A reliable smart parking management system may help drivers identify available parking spaces more efficiently and reduce the time spent searching for parking. It may also help parking operators understand occupancy patterns and make more informed operational decisions.

However, these benefits remain potential outcomes until they are measured in a real deployment. Environmental benefits, including reductions in unnecessary vehicle movement and associated emissions, should be evaluated using actual operational data rather than assumed.

Privacy and security must also be considered. Vehicle registration numbers and captured images may contain personally identifying information. Any operational deployment should therefore apply appropriate access controls, data-retention policies, and privacy safeguards. Accessibility and fairness should also be considered when designing user interfaces and allocating parking spaces.Overall, the project provides a starting point for developing more reliable and practical smart parking solutions.

REFERENCES

[1] Ultralytics, “Ultralytics YOLO Documentation.” [Online]. Available: https://docs.ultralytics.com/

[2] JaidedAI, “EasyOCR: Ready-to-Use OCR with 80+ Supported Languages.” [Online]. Available: https://github.com/JaidedAI/EasyOCR

[3] OpenCV, “OpenCV Documentation.” [Online]. Available: https://docs.opencv.org/

[4] FastAPI, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/

[5] Python Software Foundation, “Python Documentation.” [Online]. Available: https://docs.python.org/3/

[6] SQLite Consortium, “SQLite Documentation.” [Online]. Available: https://www.sqlite.org/docs.html

[7] scikit-learn Developers, “RandomForestRegressor Documentation.” [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html

[8] A. Henaien, H. Ben Elhadj, and L. Chaari Fourati, “TinyML enabled smart parking dynamic slots computing and license plate recognition,” in Proceedings of the International Wireless Communications and Mobile Computing Conference (IWCMC), 2024, pp. 1583–1588, doi: 10.1109/IWCMC61514.2024.10592456.

APPENDIX A – ALGORITHM PSEUDOCODE

A.1 Vehicle Detection Algorithm

Algorithm: DetectVehicles(frame)
Input: Image or video frame
Output: Vehicle bounding boxes, class labels, and confidence values

1. Receive the input frame from the configured camera or image source.
2. Prepare the frame in the format expected by the pretrained YOLO11n detector.
3. Run YOLO11n inference on the frame.
4. Retrieve the detected objects, bounding boxes, class labels, and confidence values.
5. Filter the detections to retain supported vehicle classes.
6. Return the filtered vehicle detections.
7. Pass the detections to the slot-occupancy logic for further processing.
8. Update the parking-slot state only when the application's occupancy logic confirms the change.

End Algorithm

A.2 Number-Plate Recognition Algorithm

Algorithm: RecognizePlate(plate_image)
Input: Vehicle or number-plate image
Output: Candidate registration text and vehicle-match result

1. Receive the number-plate image or plate crop.
2. Validate that the input image is available and suitable for processing.
3. Run the pretrained EasyOCR engine on the image.
4. Collect the candidate text returned by the OCR engine.
5. Normalize the candidate text while preserving uncertainty in the recognition result.
6. Search the vehicle records using the normalized candidate.
7. If a matching vehicle record is found, return the match.
8. Otherwise, return an unmatched result for further review.
9. Do not treat low-confidence candidate text as a verified registration number.

End Algorithm

A.3 Parking-Slot Allocation Algorithm

Algorithm: AllocateSlot(request)
Input: Booking or walk-in parking request
Output: Allocated parking slot or an unavailable result

1. Receive the parking request.
2. Validate the required request fields and booking rules.
3. Retrieve eligible parking slots from the database.
4. Filter the retrieved slots according to their current status and configured compatibility rules.

5. If no eligible slot is available, return an unavailable response.
6. Select a suitable slot according to the implemented allocation policy.
7. Recheck the selected slot's status before confirming the assignment.
8. Persist the assignment and update the relevant booking or parking-session records.
9. Update the parking-slot status consistently with the allocation.
10. Return the allocation result to the requesting interface.

End Algorithm

A.4 Parking Billing Algorithm

Algorithm: CalculateBill(session)
Input: Parking-session details
Output: Calculated bill summary and payment status

1. Receive the parking-session details.
2. Validate the recorded entry and exit timestamps.
3. Calculate the parking duration.
4. Determine the billable duration using the configured rounding rules.
5. Retrieve the configured hourly rate for the vehicle category.
6. Apply the implemented billing formula to calculate the parking charge.
7. Store the calculated amount and payment status using the supported database schema.
8. Return the bill summary to the application interface.
9. If payment confirmation is required, retain the appropriate pending status until confirmation is recorded.

End Algorithm

A.5 Parking Availability Prediction Algorithm

Algorithm: PredictAvailability(history, horizons)
Input: Available parking-history records and prediction horizons
Output: Estimated parking availability for the requested horizons

1. Receive the available parking-history records and requested prediction horizons.
2. Determine whether sufficient real historical observations are available.
3. If the real historical data is insufficient, generate or use synthetic demonstration history and identify it as synthetic.
4. Prepare the input features and target values required by the implemented Random Forest pipeline.
5. Fit or load the Random Forest regressor according to the implementation.
6. Generate availability estimates for the requested horizons, such as 30, 60, and 90 minutes.
7. Return the estimates to the application interface.
8. Display the estimates with a clear statement that prediction accuracy has not

been validated.

End Algorithm

APPENDIX B – SAMPLE OUTPUTS AND TEST EVIDENCE

The screenshots below were captured from the isolated demonstration instance. Each figure records a separate test action and timestamp; the sample records should not be combined into one simultaneous system state.

Screenshot B.1 – Admin Dashboard

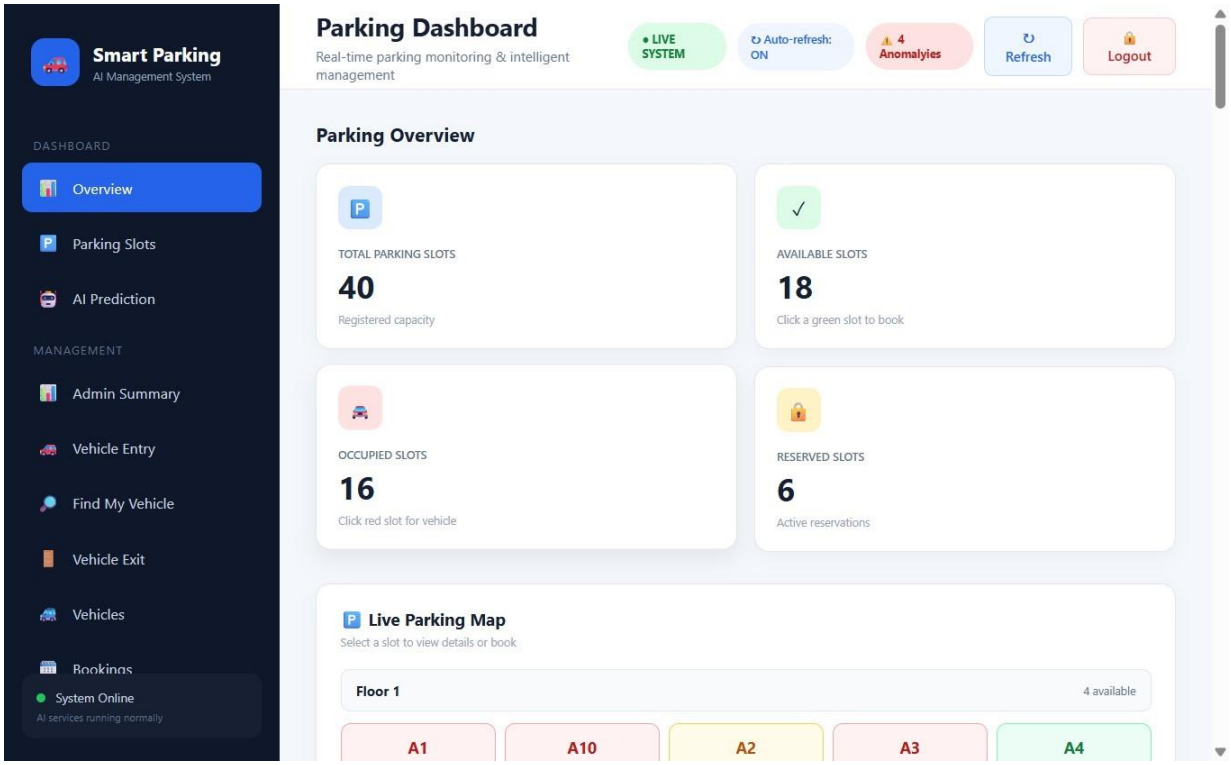


Figure B.1. Admin Dashboard.

Captured 14 October 2026, 23:57:11 IST. The demo admin overview showed 40 total slots: 18 available, 16 occupied, and 6 reserved.

Screenshot B.2 – Live Parking-Slot Status

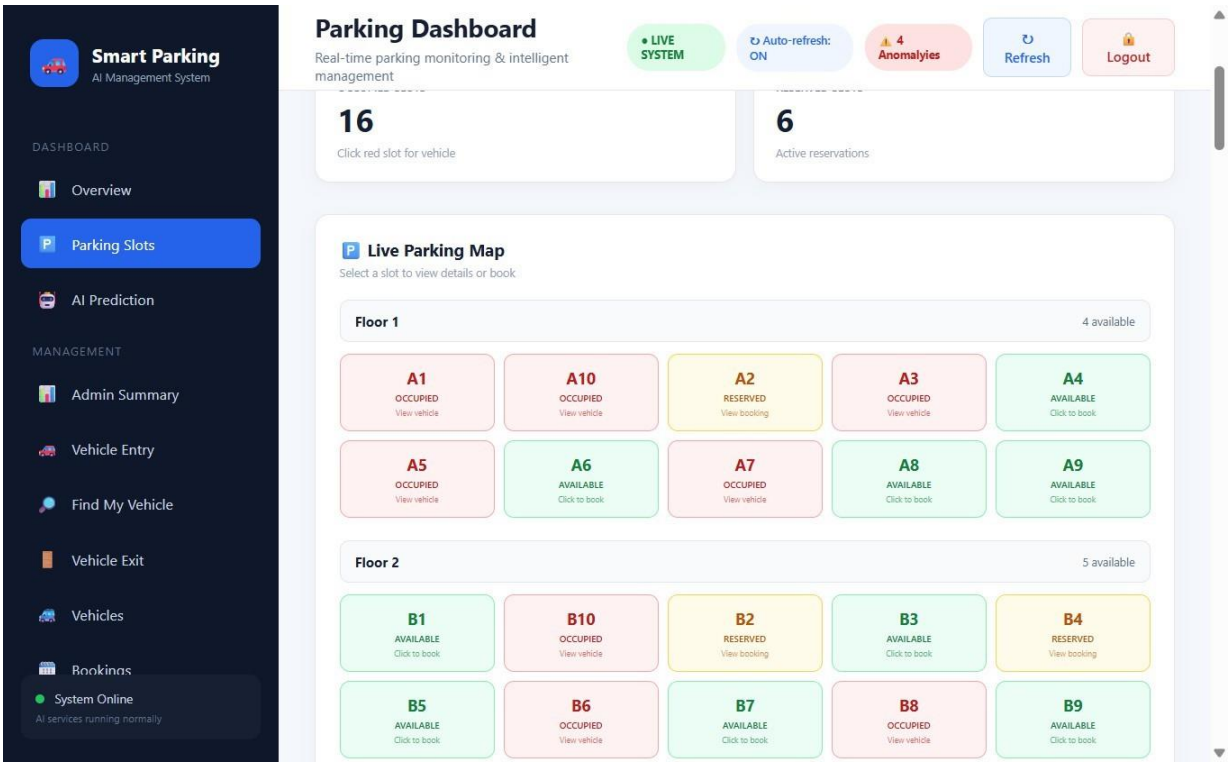


Figure B.2. Live Parking-Slot Status.

Captured 14 October 2026, 23:58:21 IST. The map shows color-coded states across floors, including reserved A2 and available A4. This is a separate snapshot from Figure B.1.

Screenshot B.3 – Vehicle Detection and Number-Plate Recognition

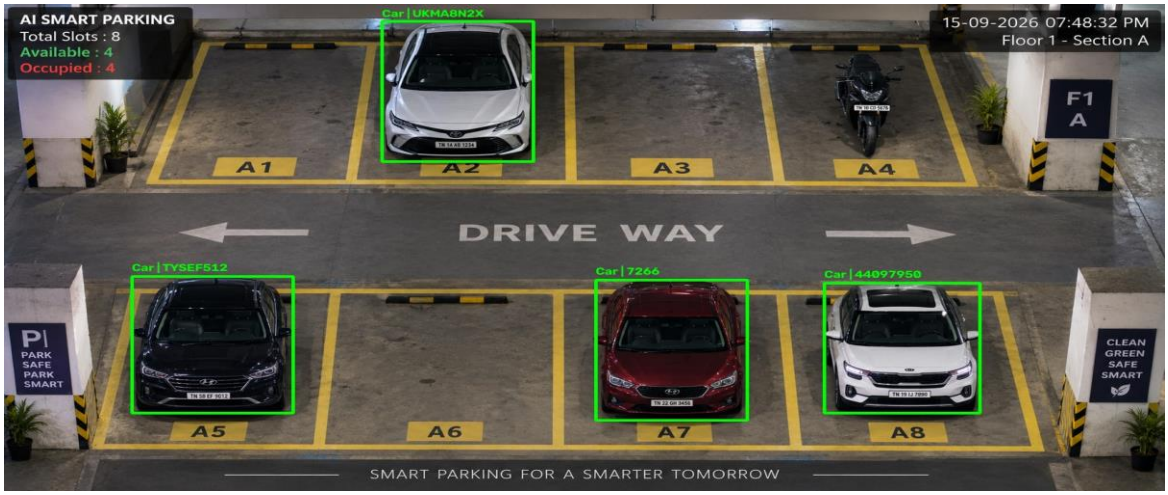


Figure B.3. Vehicle Detection and Number-Plate Recognition.

Captured 14 October 2026, 23:57:54 IST. YOLO detected four cars in camera/test_data/parking_test1.png. EasyOCR candidates were low-confidence and not verified as readable registration plates.

Screenshot B.4 – Booking Confirmation

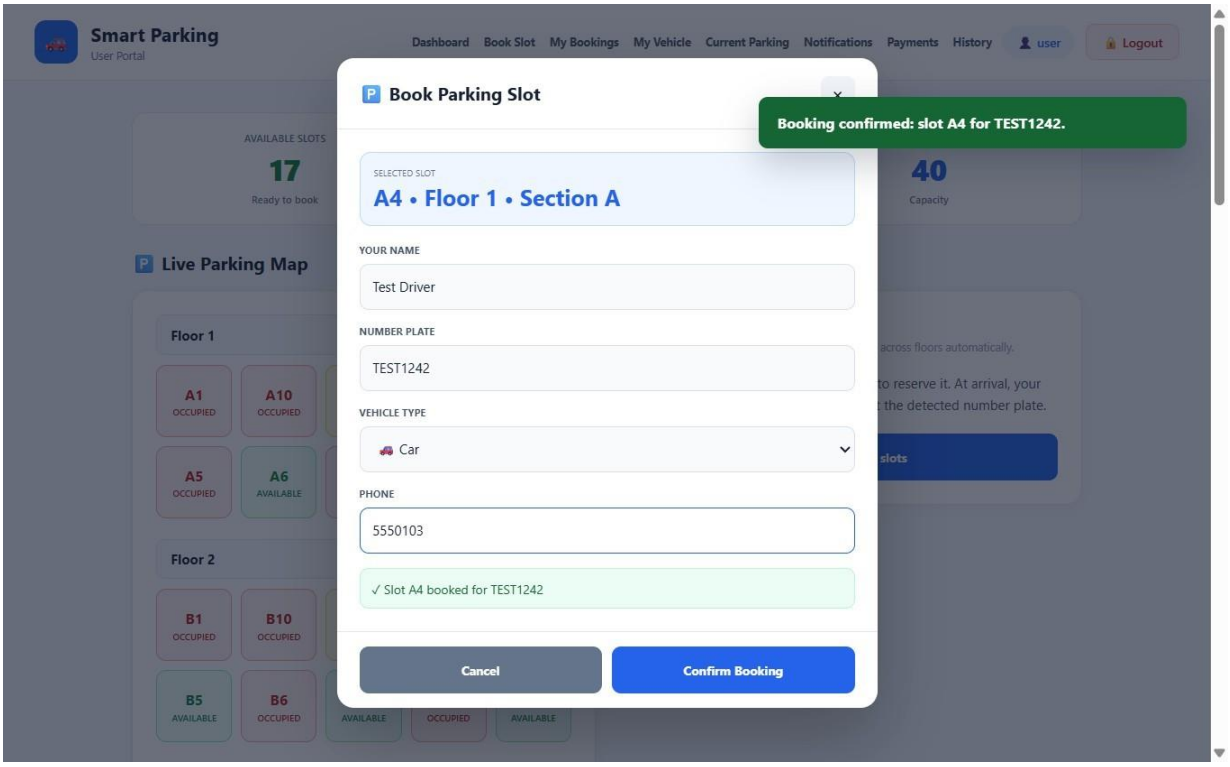


Figure B.4. Booking Confirmation.

Captured 14 October 2026, 23:59:45 IST. A car booking for A4 with plate TEST1242 and demo name Test Driver was confirmed by the user portal.

Screenshot B.5 – Walk-In Allocation

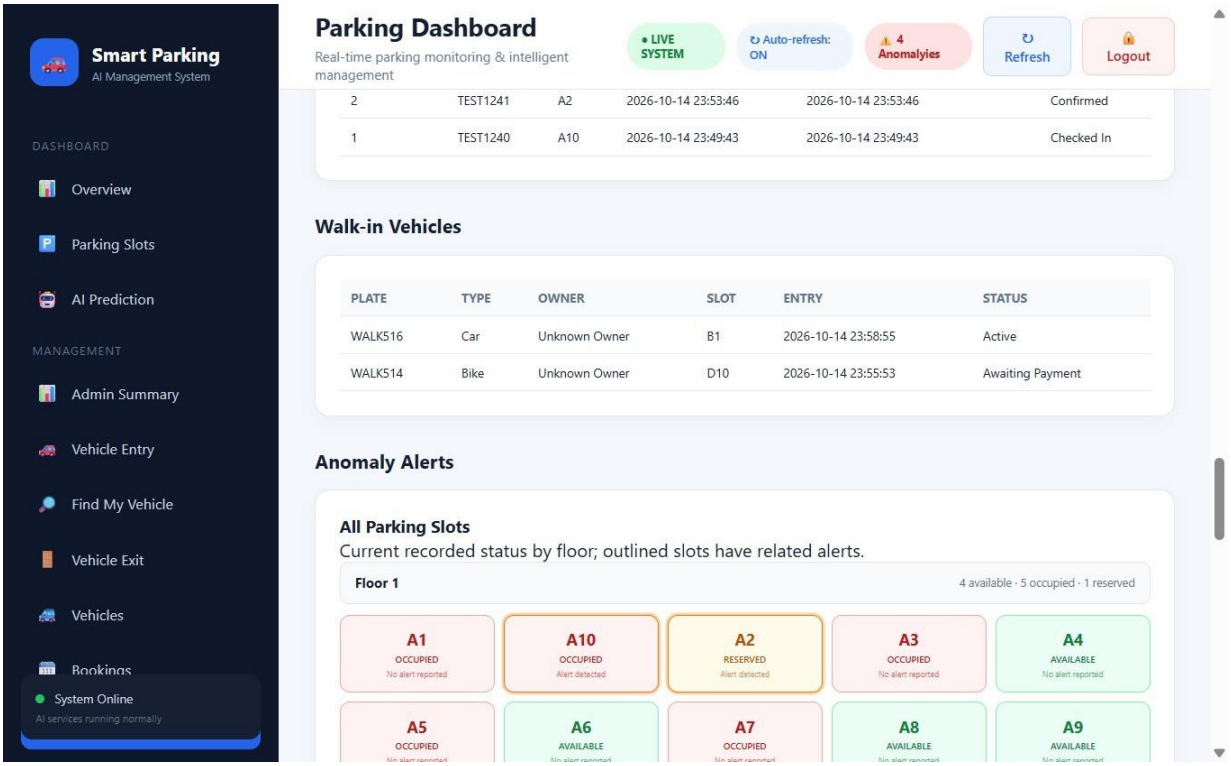


Figure B.5. Walk-In Allocation.

Captured 14 October 2026, 23:59:11 IST. The admin view showed WALK516 (Car) active in B1 and WALK514 (Bike) in D10 awaiting payment after exit.

Screenshot B.6 – Parking Session and Billing

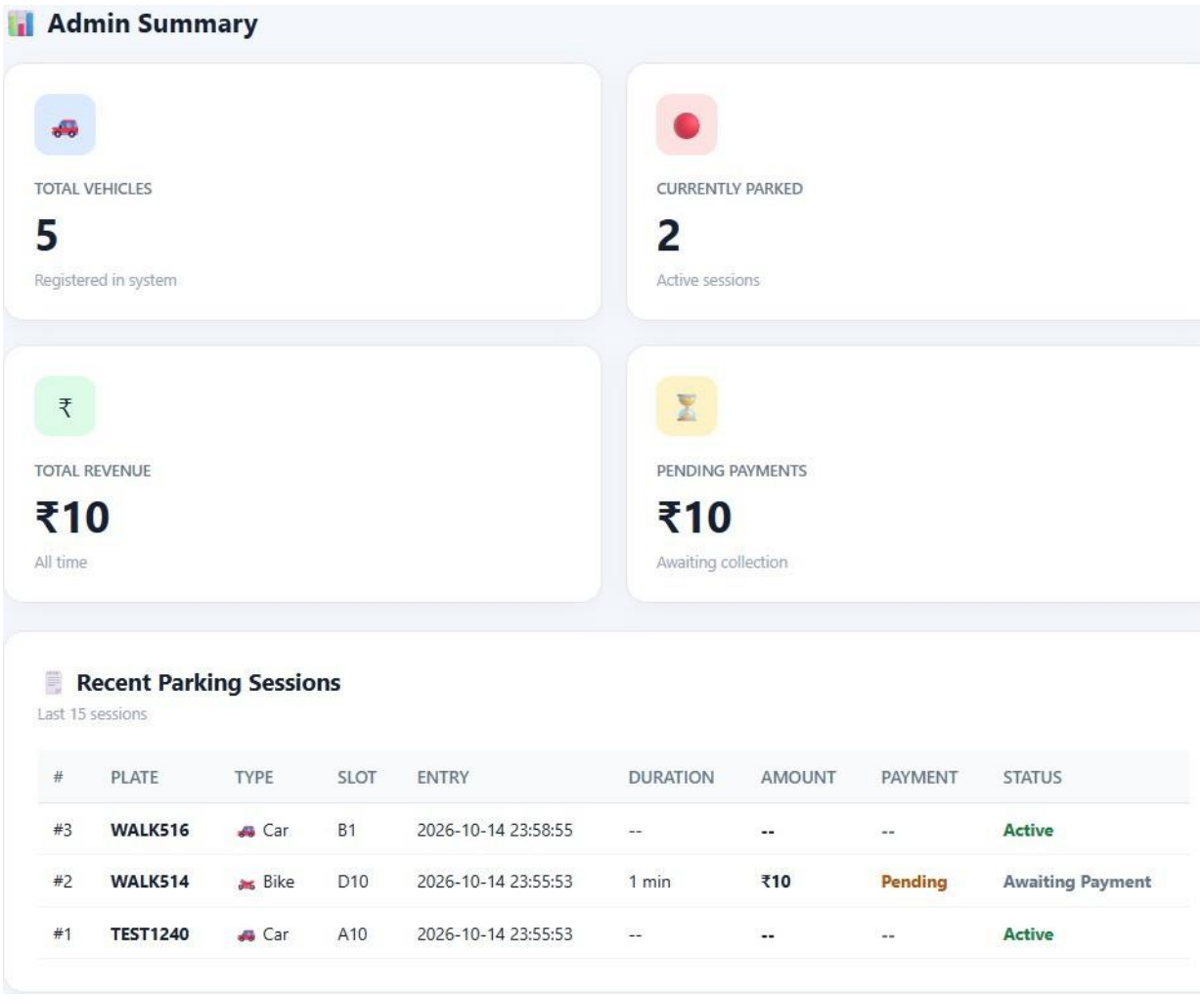


Figure B.6. Parking Session and Billing.

Captured 15 October 2026, 00:02:38 IST. WALK514 showed a one-minute duration, ₹10 amount, Pending payment, and Awaiting Payment. WALK516 remained parked in B1.

Screenshot B.7 – Prediction Output



Figure B.7. Prediction Output.

Captured 15 October 2026, 00:03:27 IST. The prediction view displayed 26 available at 30 minutes, 28 at 60 minutes, and 30 at 90 minutes. These are demo outputs, not accuracy results.

Screenshot B.8 – ROI-Based Anomaly Demonstration



Figure B.8. ROI-Based Anomaly Demonstration.

Captured 15 October 2026, 00:03:46 IST. A test API request submitted TEST1240 assigned to A10 but observed in A2, with multiple-vehicle, improper-parking, and aisle flags. The portal showed Slot Mismatch (High), Double Parking (High), Aisle Obstruction (High), and Improper Parking (Medium). This is API-supplied, not live-camera evidence

.

APPENDIX C – SOURCE CODE AND REPOSITORY

D.1 Project file map

backend/ — FastAPI routes and managers for database access, slots, bookings, entry/exit, billing, authentication, analytics, monitoring and anomalies.

camera/ — laptop-camera vehicle detection, parking-image test utilities, and number-plate recognition.

ml/ — Random Forest-based parking availability prediction.

frontend/ — browser-based administrator/user portal pages.

tests/ — route and functional tests.

D.2 Main API application

FastAPI application imports and module setup

backend/main.py

The API module imports the project managers and web framework components used by the application.

```
from fastapi import FastAPI, HTTPException, Header, Response
from fastapi.responses import FileResponse
from pydantic import BaseModel
from pathlib import Path
from datetime import datetime
from typing import Optional
import difflib
import os
import subprocess
import sys
import uuid

from backend.database import create_tables, get_connection

from backend.slot_manager import (
create_parking_slots,
get_all_slots,
get_slot,
update_slot_status
)

from backend.entry_manager import (
register_vehicle,
normalize_detected_plate,
vehicle_entry,
vehicle_exit,
find_my_car,
get_vehicle_by_slot,
get_vehicle_history
)

from backend.analytics_manager import (
get_occupancy_analytics,
log_parking_history
)
```

```
from backend.auth_manager import (
admin_login,
verify_token,
admin_logout,
user_login
)

from backend.anomaly_manager import (
get_all_anomalies
```

D.3 Database connection and schema

SQLite database path and connection

backend/database.py

SQLite is used for local persistence. Connections return rows that can be accessed by column name.

```
import sqlite3
from pathlib import Path

# Database location
BASE_DIR = Path(__file__).resolve().parent.parent
DB_PATH = BASE_DIR / "database" / "parking.db"


def get_connection():
connection = sqlite3.connect(DB_PATH)
connection.row_factory = sqlite3.Row
return connection


def create_tables():
connection = get_connection()
cursor = connection.cursor()
```

Parking-slot table definition

backend/database.py

This schema excerpt shows the persisted slot structure as defined in the source.

```
# Parking slots table
cursor.execute("""
CREATE TABLE IF NOT EXISTS parking_slots (
slot_id TEXT PRIMARY KEY,
floor INTEGER NOT NULL,
section TEXT NOT NULL,
status TEXT NOT NULL DEFAULT 'Available'
)
""")

# Bookings table
cursor.execute("""
CREATE TABLE IF NOT EXISTS bookings (
booking_id INTEGER PRIMARY KEY AUTOINCREMENT,
vehicle_id INTEGER,
slot_id TEXT,
```

```
booking_time TEXT,
start_time TEXT,
end_time TEXT,
status TEXT DEFAULT 'Confirmed',
FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),
FOREIGN KEY (slot_id) REFERENCES parking_slots(slot_id)
)
""")
```

D.4 Slot initialization and retrieval

Create sample parking slots

backend/slot_manager.py

The manager initializes example slot records and avoids inserting duplicate slot identifiers.

```
def create_parking_slots():
connection = get_connection()
cursor = connection.cursor()

# Floor 1: A1 - A10 (CCTV + Good lighting, near security)
for i in range(1, 11):
slot_id = f"A{i}"
cursor.execute("""
INSERT OR IGNORE INTO parking_slots
(slot_id, floor, section, status)
VALUES (?, ?, ?, ?)
""", (
slot_id, 1, "A",
"Occupied" if i in (1, 3, 5, 7) else "Available"
))

# Floor 2: B1 - B10 (CCTV, medium traffic)
for i in range(1, 11):
slot_id = f"B{i}"
cursor.execute("""
INSERT OR IGNORE INTO parking_slots
(slot_id, floor, section, status)
VALUES (?, ?, ?, ?)
""", (
slot_id, 2, "B",
"Reserved" if i in (2, 4) else
```

D.5 Booking workflow

Normalize vehicle registration and type

backend/booking_manager.py

Input normalization standardizes plate text and limits vehicle types to supported values.

```
def normalize_plate(number_plate):

return (
number_plate
.strip()
.upper()
.replace(" ", "")
```

```
)


def normalize_vehicle_type(vehicle_type):

allowed = {
"Car",
"Bike",
"Bus",
"Truck"
}

if not vehicle_type:
return "Car"

vehicle_type = vehicle_type.strip().title()

if vehicle_type not in allowed:
return "Car"

return vehicle_type


# ============================================================
# BOOK SLOT
```

Book a parking slot

backend/booking_manager.py

The full booking function includes validation and database operations; this listing shows its actual function signature and opening logic.

```
def book_slot(
name,
number_plate,
vehicle_type,
slot_id,
phone="",
email="",
start_time="",
end_time=""
):

number_plate = normalize_plate(
number_plate
)

vehicle_type = normalize_vehicle_type(
vehicle_type
)

slot_id = slot_id.strip().upper()

connection = get_connection()
```

```
cursor = connection.cursor()

try:

# ------------------------------------------------
# CHECK SLOT
# ------------------------------------------------

cursor.execute(
"""
SELECT *

FROM parking_slots
```

D.6 Vehicle entry and exit

Detected plate normalization

backend/entry_manager.py

The entry manager provides helpers for normalizing plate text detected by the camera workflow.

```
def normalize_detected_plate(number_plate):
plate = normalize_plate(number_plate)
if len(plate) not in {9, 10}:
return ""

digit_confusions = str.maketrans({
"O": "0",
"Q": "0",
"D": "0",
"I": "1",
"L": "1",
"Z": "2",
"S": "5",
"G": "9",
})
letter_confusions = str.maketrans({
"0": "O",
"1": "I",
"2": "Z",
"5": "S",
"6": "G",
"8": "B",
```

Vehicle entry operation

backend/entry_manager.py

This function handles the entry-side parking session workflow in the source code.

```
def vehicle_entry(
number_plate,
vehicle_type="Car"
):

number_plate = normalize_plate(
number_plate
)
```

```
vehicle_type = normalize_vehicle_type(
vehicle_type
)

vehicle = find_vehicle(
number_plate
)

# ------------------------------------------------------
# AUTO REGISTER VEHICLE
# ------------------------------------------------------

if vehicle is None:

registration = register_vehicle(
"Unknown Owner",
number_plate,
vehicle_type
)

if not registration["success"]:

return registration
```

Vehicle exit operation

backend/entry_manager.py

This function handles the exit-side workflow and is used with billing/session management.

```
def vehicle_exit(number_plate):

number_plate = normalize_plate(
number_plate
)

vehicle = find_vehicle(
number_plate
)

if vehicle is None:

return {

"success": False,

"message":
"Vehicle not registered",

"number_plate":
number_plate

}

connection = get_connection()
```

```
cursor = connection.cursor()

cursor.execute(
"""
SELECT *

FROM parking_sessions
```

D.7 Camera-based vehicle detection

YOLO vehicle detection

camera/vehicle_detection.py

The module loads the YOLO11n model and reads frames from the laptop camera. Vehicle classes are selected using the configured COCO class IDs.

```
import cv2
from ultralytics import YOLO


model = YOLO("yolo11n.pt")

camera = cv2.VideoCapture(0)

if not camera.isOpened():
print("ERROR: Could not open laptop camera.")
exit()


# COCO vehicle classes
VEHICLE_CLASSES = {
2: "Car",
3: "Motorcycle",
5: "Bus",
7: "Truck"
}


print("AI Smart Parking Vehicle Detection Started")
print("Detecting: Car, Motorcycle, Bus, Truck")
print("Press Q to exit")


while True:

success, frame = camera.read()

if not success:
break

results = model(
frame,
verbose=False
)

vehicle_count = 0
```

for result in results:

boxes = result.boxes

for box in boxes:

class_id = int(box.cls[0])
confidence = float(box.conf[0])

if class_id in VEHICLE_CLASSES:

vehicle_count += 1

x1, y1, x2, y2 = map(

D.8 Number-plate recognition

Load detection/OCR models and clean OCR text

camera/plate_recognition.py

The plate-recognition module uses Ultralytics YOLO and EasyOCR, then removes non-alphanumeric characters from recognized text.

```
import cv2
import easyocr
import re
from pathlib import Path
from ultralytics import YOLO


# ============================================================
# 1. LOAD MODELS
# ============================================================

print("Loading YOLO...")

model = YOLO("yolo11n.pt")

print("Loading EasyOCR...")

reader = easyocr.Reader(
['en'],
gpu=False
)

print("Models loaded successfully.")


# ============================================================
# 2. CLEAN OCR TEXT
# ============================================================

def clean_plate(text):

text = text.upper()
```

```
text = re.sub(
r'[^A-Z0-9]',
'',
text
)

return text


# ============================================================
# 3. OCR FUNCTION
# ============================================================

def read_plate_from_crop(vehicle_crop):

if vehicle_crop is None:
return None

if vehicle_crop.size == 0:
```

D.9 Parking availability prediction

Read historical parking records

ml/parking_prediction.py

The prediction module attempts to retrieve historical parking information from the SQLite database.

```
def get_history():

connection = get_connection()

query = """
SELECT
timestamp,
day_of_week,
hour,
total_slots,
occupied_slots,
available_slots,
entry_count,
exit_count
FROM parking_history
ORDER BY timestamp
"""

data = pd.read_sql_query(
query,
connection
)

connection.close()

return data
```

```
# ============================================================
# CREATE DEMO TRAINING DATA
# ============================================================
```

Random Forest model setup

ml/parking_prediction.py

The model is based on scikit-learn's RandomForestRegressor. The complete file contains the feature preparation and prediction flow.

```
y = history["available_slots"]


# ----------------------------------------------------------
# Random Forest
# ----------------------------------------------------------

model = RandomForestRegressor(

n_estimators=150,

random_state=42,

max_depth=10

)


model.fit(
X,
y
)


print(
"ML model trained successfully!"
)


return model


# ============================================================
```

D.10 Billing and rate calculation

Vehicle-type hourly rates and billing formula

backend/billing_manager.py

The source defines different base hourly rates by vehicle type, rounds parking time up to billable hours, and applies a higher rate after the first two billable hours.

```
from math import ceil


HOURLY_RATES = {
"Bike": 10,
```

```
"Car": 20,
"Bus": 40,
"Truck": 40,
}


def calculate_dynamic_bill(vehicle_type, duration_minutes):
base_rate = HOURLY_RATES.get(vehicle_type, HOURLY_RATES["Car"])
hours = max(1, ceil(max(duration_minutes, 1) / 60))
standard_hours = min(hours, 2)
extended_hours = max(0, hours - 2)

amount = (
standard_hours * base_rate
+ extended_hours * base_rate * 1.5
)

return {
"amount": round(amount, 2),
"billable_hours": hours,
"hourly_rate": base_rate,
"vehicle_type": vehicle_type,
}
```

D.11 Parking anomaly/alert management

Detect expired/no-show bookings

backend/anomaly_manager.py

This is a source excerpt from the anomaly manager. It checks confirmed bookings against arrival/session records.

```
def detect_no_show_bookings(expiry_minutes=30):

connection = get_connection()
cursor = connection.cursor()

cutoff = (
datetime.now()
.__class__.now()
)

cursor.execute("""
SELECT
b.booking_id,
b.slot_id,
b.booking_time,
v.number_plate,
u.name AS owner_name
FROM bookings b
JOIN vehicles v ON b.vehicle_id = v.vehicle_id
LEFT JOIN users u ON v.user_id = u.user_id
WHERE b.status = 'Confirmed'
AND NOT EXISTS (
SELECT 1 FROM parking_sessions ps
WHERE ps.vehicle_id = b.vehicle_id
```

```
AND ps.slot_id = b.slot_id
AND ps.status = 'Active'
)
""")

bookings = cursor.fetchall()
connection.close()

anomalies = []

Record vision observations

backend/monitoring_manager.py

Monitoring logic records observations from the vision workflow for later alert review.

def record_vision_observation(observation):
number_plate = observation.get("number_plate", "").strip().upper().replace(" ", "") or None
slot_id = observation.get("slot_id", "").strip().upper() or None
alerts = []

if number_plate and slot_id:
connection = get_connection()
session = connection.execute(
"""
SELECT slot_id FROM parking_sessions
WHERE status = 'Active' AND vehicle_id = (
SELECT vehicle_id FROM vehicles WHERE number_plate = ?
)
ORDER BY session_id DESC LIMIT 1
""",
(number_plate,)
).fetchone()
connection.close()
if session and session["slot_id"] != slot_id:
alerts.append((
"Slot Mismatch",
f"{number_plate} is assigned to {session['slot_id']} but detected in {slot_id}.",
"High",
session["slot_id"],
))

checks = (
("vehicle_count", lambda value: value > 1, "Double Parking", "Multiple vehicles are detected in one parking slot.", "High"),
```

D.12 Frontend integration

User portal structure and API calls

frontend/user.html

The user portal is a browser-based HTML page. The excerpt shows its document structure and selected API-call locations from the uploaded file.

```
1: <!DOCTYPE html>
2: <html lang="en">
3: <head>
4: <meta charset="UTF-8">
```

5: <meta name="viewport" content="width=device-width, initial-scale=1.0">
6: <title>Smart Parking — User Portal</title>
7: <style>
8: *{ margin:0; padding:0; box-sizing:border-box; font-family:"Segoe UI",Arial,sans-serif; }
9: html{ scroll-behavior:smooth; }
10: body{ background:#f4f7fb; color:#172033; }
11:
12: #loginPage{
585: btn.textContent='Signing in...'; btn.disabled=true;
586: try{
587: const res = await fetch('/api/auth/user/login',{
588: method:'POST', headers:{'Content-Type':'application/json'},
589: body: JSON.stringify({username:u, password:p})
590: });
591: const data = await res.json();
607:
608: async function doLogout(){
609: if(authToken) await fetch('/api/auth/logout',{method:'POST',headers:{'x-token':authToken}}).catch(()=>{});
610: authToken=null; sessionStorage.clear();
611: document.getElementById('loginPage').style.display='flex';
612: document.getElementById('loginUsername').value='';
613: document.getElementById('loginPassword').value='';
618: if(!authToken) return;
619: try{
620: const res = await fetch('/api/auth/verify',{headers:{'x-token':authToken}});
621: if(res.ok){
622: const d = await res.json();
623: if(d.role !== 'user'){
624: authToken=null;
642: async function loadLatestCameraDetection(){
643: try {
644: const response = await fetch(`/api/camera/latest?ts=${Date.now()}`, { cache: 'no-store' });
645: if (!response.ok) return;
646: const data = await response.json();
647: if (!data || !data.number_plate) return;
648:
689: async function loadStats(){
690: try{
691: const r = await fetch('/api/statistics');
692: const d = await r.json();
693: document.getElementById('uAvailable').textContent = d.available;
694: document.getElementById('uOccupied').textContent = d.occupied;
695: document.getElementById('uTotal').textContent = d.total;

D.13 API route tests

Test setup and billing checks

tests/test_routes.py

The test module imports the FastAPI test client and project managers, then defines functional tests for billing and route behavior.

import unittest
import os
import subprocess

```
import sys
from pathlib import Path
from tempfile import TemporaryDirectory
from unittest.mock import MagicMock, patch

from fastapi.testclient import TestClient

from backend.billing_manager import calculate_dynamic_bill
from backend.booking_manager import book_slot
from backend.database import create_tables, get_connection
from backend.entry_manager import normalize_detected_plate, vehicle_entry, vehicle_exit
from backend.anomaly_manager import get_all_anomalies
from backend.main import app
from backend.monitoring_manager import get_open_alerts, record_alert, record_vision_observation
from backend.slot_manager import create_parking_slots


class DynamicBillingTests(unittest.TestCase):
def test_vehicle_type_changes_base_rate(self):
bike_bill = calculate_dynamic_bill("Bike", 60)
car_bill = calculate_dynamic_bill("Car", 60)

self.assertEqual(bike_bill["amount"], 10)
self.assertEqual(car_bill["amount"], 20)

def test_extended_parking_uses_hourly_rate_only(self):
bill = calculate_dynamic_bill("Car", 13 * 60)

self.assertEqual(bill["billable_hours"], 13)
self.assertEqual(bill["amount"], 370)


class ParkingRouteTests(unittest.TestCase):
def test_detected_plate_normalizer_accepts_one_letter_series(self):
self.assertEqual(normalize_detected_plate("MP33 C 3370"), "MP33C3370")
self.assertEqual(normalize_detected_plate("TNOGBY9726"), "TN09BY9726")
self.assertEqual(normalize_detected_plate("FEES6"), "")

def test_camera_entry_registers_plate_and_exit_rejects_unknown_plate(self):
with TemporaryDirectory() as temporary_directory:
database_path = Path(temporary_directory) / "parking.db"
with (
patch("backend.database.DB_PATH", database_path),
patch("backend.slot_manager.DB_PATH", database_path),
):
create_tables()
create_parking_slots()
entry = vehicle_entry("AUTO123", "Car")
unknown_exit = vehicle_exit("UNKNOWN123")
connection = get_connection()
vehicle = connection.execute(
"SELECT number_plate FROM vehicles WHERE number_plate = ?",
```

D.14 Setup and dependencies

Project run notes

README.md

The excerpt is taken from the project README. Follow the complete README for all environment and run instructions.

AI Smart Parking

FastAPI, SQLite, YOLO11, and EasyOCR parking workflow with separate admin and user portals.

Run

```powershell
python -m pip install -r requirements.txt
python -m pip install --force-reinstall --no-deps opencv-python==4.11.0.86
python -m uvicorn backend.main:app --reload
```

The camera preview requires GUI-enabled OpenCV. Reinstalling this wheel after the requirements is important because EasyOCR also depends on the headless OpenCV package, which does not support `cv2.imshow`.

Open `/dashboard` for the admin portal, `/user` for the user portal, or `/docs` for the API. Demo credentials seeded into an empty database are `admin / admin123` and `user / user123`.

Run the unit tests with `python -m pytest`; standalone camera demos are not collected as tests.

Parking Flow

1. `camera/plate_camera.py` runs YOLO vehicle detection before EasyOCR. Press `E` to switch the gate between entry and exit. Entry verifies a confirmed plate booking and otherwise records a walk-in. Exit creates a bill; the bay is released only after payment confirmation.
2. `camera/parking_detection.py` updates unassigned bay occupancy and reports double parking, bay overlap, and aisle obstruction to the backend. Calibrate `parking_slots` and `AISLE_ZONES` to the camera image before using alerts operationally. Use separate camera devices or configure the capture index if running both camera scripts at once.
3. Alerts and in-app notifications are persisted in SQLite. Slot mismatch observations use `POST /api/vision/observations` with `number_plate` and the detected `slot_id`.

Walk-in allocation chooses an available bay on the floor with the fewest occupied bays. Confirmed bookings keep their reserved bay. Slot updates from vision do not overwrite confirmed bookings or active/pending sessions.

Billing

Base hourly rates in INR are Bike 10, Car 20, Bus 40, and Truck 40. The first two billable hours use the base rate; later hours use 1.5x. `/api/billing/rates` exposes the current rates and policy. Payment confirmation is currently a demo action, not a payment-provider integration.

Main Records

SQLite stores users, vehicles, parking slots, bookings, parking sessions, parking history, payments, admin credentials, parking alerts, and user notifications. Legacy safety-score fields are ignored by the

API and are not created for new databases.

Python dependencies

requirements.txt

These packages are listed by the uploaded project as its Python dependencies.

fastapi
uvicorn
pydantic
numpy
pandas
scikit-learn
easyocr
requests
ultralytics
pytest
opencv-python==4.11.0.86

D.15 Source package note

The excerpts above are intentionally shortened to keep this appendix within the requested length. They do not replace the complete source files. The full source package should be submitted separately as the project ZIP and maintained in the TEAM-12 GitHub repository.

Implementation note: the YOLO and EasyOCR components use pretrained models; the parking prediction module uses scikit-learn Random Forest; anomaly/monitoring behavior should be described according to the implemented rules and checks in the complete sou

APPENDIX D — CONFERENCE REGISTER

Gmail SARAVANAN <saravananvengadesin@gmail.com>

International Conference on Advances in Biosciences, Data Science and AI 2026 : Submission (246) has been edited.

2 messages

Microsoft CMT <noreply@msr-cmt.org> Fri, Oct 9, 2026 at 10:19 AM
Reply-To: Microsoft CMT - Do Not Reply <noreply@msr-cmt.org>
To: saravananvengadesin@gmail.com

Hello,

The following submission has been edited.

Track Name: ICABDAI2026

Paper ID: 246

Paper Title: AI-Driven Smart Parking Management

Abstract:
The increasing number of vehicles in urban and multi-level parking facilities creates challenges in parking-space utilization, vehicle identification, reservation management, and parking supervision. This paper presents an AI-Driven Smart Parking Management System that integrates computer vision, automatic number plate recognition, dynamic slot allocation, parking anomaly detection, predictive analytics, and parking-session management into a unified framework. The proposed system uses YOLO11n for vehicle detection and EasyOCR for automatic number plate recognition (ANPR) at vehicle entry and exit. Recognized vehicle numbers are verified against active bookings, while vehicles without reservations are handled through a walk-in mechanism and assigned an available and unreserved parking slot dynamically. During the parking session, an overhead camera and predefined regions of interest are used to identify slot mismatch, improper or double parking, and driving-aisle obstruction. A Random Forest model is incorporated to predict parking availability for 30, 60, and 90 minutes using parking-history information. At vehicle exit, the system calculates parking charges using configurable duration- and violation-based billing rules and updates the corresponding parking session and slot status. The system is implemented using Python, FastAPI, SQLite, OpenCV, YOLO11n, and EasyOCR, with separate Admin and User portals for parking monitoring, booking, vehicle management, anomaly alerts, billing, prediction, and notifications. The implemented Phase 1 prototype demonstrates the complete workflow from vehicle entry and slot allocation to parking monitoring, vehicle exit, billing, and slot release, while large-scale quantitative validation using real-world labelled parking data and physical sensing hardware remains part of future development.

Created on: Tue, 06 Oct 2026 04:39:06 GMT

Last Modified: Fri, 09 Oct 2026 04:49:48 GMT

Authors:
- saravananvengadesin@gmail.com (Primary)
- ragul20060317@gmail.com
- lyyappanmaharaja31@gmail.com

Primary Subject Area: AI - Intelligent Systems & Automation

Secondary Subject Areas: Not Entered

Submission Files:
Smart parking AI Check1.pdf (1 Mb, Tue, 06 Oct 2026 04:42:19 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

Microsoft CMT <noreply@msr-cmt.org> Fri, Oct 9, 2026 at 10:22 AM
Reply-To: Microsoft CMT - Do Not Reply <noreply@msr-cmt.org>
To: saravanananvengadesin@gmail.com

Hello,

The following submission has been edited.

Track Name: ICABDAI2026

Paper ID: 246

Paper Title: AI-Driven Smart Parking Management

Abstract:
The increasing number of vehicles in urban and multi-level parking facilities creates challenges in parking-space utilization, vehicle identification, reservation management, and parking supervision. This paper presents an AI-Driven Smart Parking Management System that integrates computer vision, automatic number plate recognition, dynamic slot allocation, parking anomaly detection, predictive analytics, and parking-session management into a unified framework. The proposed system uses YOLO11n for vehicle detection and EasyOCR for automatic number plate recognition (ANPR) at vehicle entry and exit. Recognized vehicle numbers are verified against active bookings, while vehicles without reservations are handled through a walk-in mechanism and assigned an available and unreserved parking slot dynamically. During the parking session, an overhead camera and predefined regions of interest are used to identify slot mismatch, improper or double parking, and driving-aisle obstruction. A Random Forest model is incorporated to predict parking availability for 30, 60, and 90 minutes using parking-history information. At vehicle exit, the system calculates parking charges using configurable duration- and violation-based billing rules and updates the corresponding parking session and slot status. The system is implemented using Python, FastAPI, SQLite, OpenCV, YOLO11n, and EasyOCR, with separate Admin and User portals for parking monitoring, booking, vehicle management, anomaly alerts, billing, prediction, and notifications. The implemented Phase 1 prototype demonstrates the complete workflow from vehicle entry and slot allocation to parking monitoring, vehicle exit, billing, and slot release, while large-scale quantitative validation using real-world labelled parking data and physical sensing hardware remains part of future development.

Created on: Tue, 06 Oct 2026 04:39:06 GMT

Last Modified: Fri, 09 Oct 2026 04:52:26 GMT

Authors:
- saravanananvengadesin@gmail.com (Primary)
- ragul20060317@gmail.com
- Iyyappanmaharaja31@gmail.com

Primary Subject Area: AI - Intelligent Systems & Automation

Secondary Subject Areas: Not Entered

Submission Files:
Smart parking AI Check1.pdf (1 Mb, Tue, 06 Oct 2026 04:42:19 GMT)
Smart Parking abstract Final.docx (91 Kb, Fri, 09 Oct 2026 04:52:05 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.
[Quoted text hidden]